THÈSE

Pour obtenir le grade de

DOCTEUR DE L'UNIVERSITÉ GRENOBLE ALPES

École doctorale : EEATS - Electronique, Electrotechnique, Automatique, Traitement du Signal
Spécialité : Nano électronique et Nano technologies
Unité de recherche : CEA /LIST - Laboratoire d'intégration de systèmes et de technologies

Conception d'un Système Mémoire pour Traitement de Données Creuses

Design of a Memory System for Sparse Data Processing

Présentée par :

Valentin ISAAC--CHASSANDE

Direction de thèse :

Frédéric ROUSSEAU PROFESSEUR DES UNIVERSITES, Grenoble INP - UGA	Directeur de thèse
Adrian EVANS CEA	Co-encadrant de thèse

Rapporteurs :

Thèse soutenue publiquement le , devant le jury composé de :

Valentin ISAAC--CHASSANDE
Université Grenoble Alpes
CEA Grenoble (DRT/LIST/DSCIN/LSTA)
TIMA Laboratory (SLS)
ORCID : 0009-0007-9842-5777
valentin.isaacchassande@univ-grenoble-alpes.fr
valentin.isaacchassande@cea.fr

THESIS

DESIGN OF A MEMORY SUB-SYSTEM FOR SPARSE DATA PROCESSING

October 29, 2025

Thesis Adviser: Pr. Frédéric ROUSSEAU
Thesis Co-Supervisors: Dr. Adrian EVANS
Dr. Yves DURAND

EEATS Doctoral College, NENT Speciality
2022 – 2025 Academic Years
From 10.25.2022 to 10.24.2025

Acknowledgements

TODO

"Work. Or school. This is for the money. But what is anything else, for that matter, but required means of enhancing your bare living – better sleep and better food – like spending decades just to turn a little knob higher on a pathetic radio called existence."

Karl Kristian Flores, The Goodbye Song.

Résumé

TODO

Abstract

TODO

List of Publications

This thesis is based on the following publications:

I Valentin Isaac--Chassande, Adrian Evans, Yves Durand, and Frédéric Rousseau. 2024. Dedicated Hardware Accelerators for Processing of Sparse Matrices and Vectors: A Survey. *ACM Trans. Archit. Code Optim.* 21, 2, Article 27 (Feb. 2024), 26 pages. Reference [26]

II Valentin Isaac--Chassande, Adrian Evans, Yves Durand, and Frédéric Rousseau. 2024. SpDCache: Region-Based Reduction Cache for Outer-Product Sparse Matrix Kernels. In *2024 IEEE 35th International Conference on Application-specific Systems, Architectures and Processors (ASAP)*. IEEE Computer Society, Los Alamitos, CA, USA, 3–7. Reference [27]

III **TODO**

The papers will be referred to in the thesis using their Roman numerals.

Summary

Notations

$$\forall (a,b) \in \mathbb{Z}^2, [\![a,b]\!] = [a,b] \cap \mathbb{Z}$$

nnz, M, A, b and c, $\overline{nnz_r}$, d_r ...

TODO

Glossary

band Refers to an area of a matrix that is concentrated around the diagonal. 9, 16, 18, 35, 37, 38, 40

banded Refers to a matrix structure where the non-zero elements are concentrated around the diagonal. 16, 18, 26, 29, 31, 35, 37, 40, 50

bandwidth With this typography, refers to the size of the band of a matrix. 16

common rank Refers to a rank that is common between two data-entries, when computing a kernel. 18, 19, 20, 23, 25

dense format Is the uncompressed format where all zeros and non-zeros are stored, without any use of metadata. 17, 23, 24

eviction Is a cache event where a valid cache entry is flushed and freed. 32, 33, 34, 35, 36, 37, 40

fiber Is the generic term to describe a one-dimensional stream of data or metadata. It corresponds to a row or column in the case of a two-dimensional matrix. A stream of indices is a *fiber* of metadata. 22, 25

gather Refers to reading data from memory. We specifically use this term to categorize data transfers that are irregular. 25, 29

hit Is a cache event where a given address does match one of the valid cache entries. 32, 33, 34, 36, 37

ineffectual operation Is an operation that leads to a neutral result, 0 for an addition or 1 for a multiplication. 22

intersection Refers to the process of finding non-zero partial sums, when computing a matrix multiplication. 19, 20, 21, 22, 23, 25, 26, 29, 52

irregular access See "random" access. 9, 15, 29, 31, 32

irregular data Refers to data that has low spatial and temporal locality, thus being more likely to generate irregular accesses. 29, 32

metadata Is data that provides information about other data, such as index arrays of sparse formats. 9, 10, 17, 18, 22, 25, 44

miss Is a cache event where a given address does not match any of the valid cache entries. 22, 26, 32, 33, 34, 36, 37

partial sum Refers to the intermediary result before updating the output, when computing a matrix multiplication. 9, 19, 22, 23, 25

'perfectly' banded Refers to a matrix structure where all the non-zero elements are concentrated within a delimitated area around the diagonal (the band). 16

"random" access Is a memory access that occurs on an arbitrary memory address (not necessarily conscutive in memory with previous and next accesses). 9, 10, 15

"random" update Is a "random" access where the access is a read, modify and write on a given address. It is similar to a "random" *reduction*. 22

rank Refers to a dimension of the kernel. 9, 17, 18, 19, 20, 21, 22, 25

reduction Refers to the process of updating an entry. 10, 19, 23, 24, 25, 26, 29, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 50, 52

reduction cache Is a data-cache supporting *reduction* operations. 33, 34, 35, 37, 38, 40

scatter Refers to updating data in memory. We specifically use this term to categorize data transfers that are irregular. 25, 29

sequential accesses Are a group of memory accesses (from a data array, for example) that occur on conscutive memory addresses, in order through time. 17, 32

set Refers to a fixed number of cache-lines, a subdivision related to set-associative caches. 10, 32, 34, 35, 37, 39

sparse format Is a compression format for sparse matrices, which avoids storing zeros by using metadata. 9, 17, 18, 20, 21, 22, 23, 29, 33, 44

tag Refers to cache metadata that allows identification of cache-line memory addresses. 32

tile Refers to a sub-region of a *tiling* process. 20, 21, 22, 26

tiling Is a method consisting in splitting data into several independent sub-regions to be processed separately. 10, 20, 21, 22, 25, 26, 29, 44, 50, 52

way Refers to the number of cache-lines that a *set* contains. 32, 34, 37

Acronyms

BaCSC *Banded Compressed Sparse Column*. 18, 29, 50

BCSR *Blocked Compressed Sparse Row*. 17

COO *COOrdinate*. 17, 18, 24, 29, 44, 50

CSC *Compressed Sparse Column*. 17, 18, 22, 23, 24, 25, 29, 33, 44, 50, 52

CSR *Compressed Sparse Row*. 17, 18, 21, 22, 23, 24, 25, 29, 44, 50, 52

DIA *DIAgonal*. 17

DRC *Dense Region Cache*. 35, 36, 37, 38, 39, 40, 50

ELL *ELLPACK*. 17

GC Generic Cache. 37, 38, 39

GRC *Generic Region Cache*. 35, 37, 38, 39, 40, 50

`Gust` *Gustavson*. 18, 20, 21, 25, 26, 28, 29, 51, 52

HYB *HYBbrid*. 17

IBR *In-Band Region*. 35, 36, 37, 40

`IP` *Inner-Product*. 18, 19, 20, 21, 22, 23, 25, 26, 28, 29, 51, 52

LRU *Least Recently Used*. 32, 37

MM Matrix-Matrix. 18, 19, 20, 21, 25, 29, 52

MV Matrix-Vector. 18, 19, 20, 21, 25, 29, 52

OBR *Out-of-Band Region*. 35, 36, 37, 40

`OP` *Outer-Product*. 18, 19, 20, 21, 22, 23, 24, 25, 26, 28, 29, 31, 33, 34, 35, 37, 38, 50, 51, 52

PO *partial output*. 19, 20, 21, 22, 23, 25, 29

RedC Reduction Cache. 37, 38, 39

RMW *Read-Modify-Write*. 36, 37, 38

RMWQ *Read-Modify-Write Queue*. 36, 37

SpDC SpDCache. 31, 35, 36, 37, 38, 39, 40, 50

SpGEMM General Sparse Matrix-Matrix multiplication. 18

SpGEMV General Sparse Matrix-Vector multiplication. 18

SpMM Sparse Matrix-Matrix multiplication. 18

SpMSpM Sparse Matrix-Sparse Matrix multiplication. 18, 19, 20, 22, 23, 24, 25, 26, 28, 29, 51, 52

SpMSpV Sparse Matrix-Sparse Vector multiplication. 18

SpMV Sparse Matrix-Vector multiplication. 18, 19, 20, 21, 28, 29, 31, 33, 34, 35, 37, 38, 50, 52

SRT *Sparse Region Table*. 35, 36, 37, 38, 39, 40, 50

Introduction

THE rapid growth of data-intensive applications has made sparse and irregular data structures increasingly prevalent in scientific computing, machine learning, and graph analytics. However, efficiently processing sparse matrices remains a fundamental challenge due to their inherent irregularity, which leads to unpredictable memory access patterns, limited opportunities for data reuse, and suboptimal utilization of hardware resources. Existing hardware accelerators have demonstrated partial solutions, yet they often rely on restrictive assumptions, exhibit limited scalability, or fail to fully exploit the memory bandwidth.

This thesis addresses the following central question:

PROBLEM STATEMENT

How can hardware and architectural mechanisms be designed to efficiently support computation with irregular sparse data, in particular random reductions, while ensuring scalability, adaptability, and high performance across diverse workloads?

To tackle this problem, we investigate the limitations of existing algorithms and accelerators, propose a novel cache architecture with support for reduction operations, tailored to sparse data. We then evaluate the behaviour of this specialized cache through modeling, simulation, and hardware prototyping.

This manuscript is structured around several key contributions. We begin with an in-depth discussion of irregular data, with a particular emphasis on sparse matrices (Chapter 1). This includes a study of fundamental sparse matrix multiplication kernels and their associated algorithms, highlighting the main hardware challenges that arise when dealing with sparse data. A high-level analysis of algorithms for sparse data computing is provided, with a focus on basic storage schemes and opportunities for data reuse. Building on this foundation, we review the state-of-the-art in hardware accelerators designed for efficient sparse matrix computing (Chapter 2). This review identifies common characteristics, proposes a classification of existing accelerators and design tendencies, offers a comparative analysis, and identifies the major tendencies across designs. This analysis of the state of the art guided our subsequent architectural choices.

Following this survey, we introduce a novel cache architecture specifically designed to address the challenge of "random" reductions (Chapter 3). The proposed cache integrates reduction support and organizes its memory into multiple regions optimized for denser and sparser data regions. A Python model was developed to simulate memory transactions in this cache and to compare its behavior with that of a generic cache. To further investigate its performance, we complement this analysis with a probabilistic model, implemented in C, that enables rapid simulation and sizing parameters exploration depending on matrices structure (Chapter **??**). At the micro-architectural level, we detail the virtual partitioning of cache memory into multiple configurable regions, which ensures both flexibility and adaptability to diverse workloads (Chapter **??**).

The proposed cache is then evaluated through RTL simulation, which reveals two distinct classes of matrices that exhibit different performance profiles (Chapter **??**). Finally, the manuscript outlines potential future enhancements to improve both performance and scalability, including a multicore extension of the cache architecture and optimizations targeting bandwidth efficiency (Chapter **??**).

Contributions of the Thesis

The main contributions of this thesis can be summarized as follows:

1. **Analysis of irregular and sparse data computing** (Chapter 1)
   - Study of fundamental sparse matrix multiplication kernels and algorithms.
   - Identification of the key hardware challenges associated with sparse data processing.
   - High-level analysis of algorithms for sparse data computing, with emphasis on storage schemes and data reuse opportunities.
2. **Survey and classification of state-of-the-art accelerators** (Chapter 2)
   - Comprehensive review of hardware accelerators for sparse matrix computing.
   - Identification of common architectural features and design tendencies.
   - Comparative analysis to explain design choices under varying objectives and constraints.
3. **Design of a cache architecture for random reductions** (Chapter 3)
   - Proposal of a cache with native reduction support, capable of handling both dense and sparse data via multiple cache regions.
   - Development of a Python-based simulation model to analyze memory transactions and benchmark against a generic cache.
4. **Probabilistic modeling and exploration of cache behavior** (Chapter **??**)
   - Implementation of a lightweight C-based probabilistic model for rapid simulation.
   - Exploration of cache dimensioning and behavior across diverse workloads.
5. **Micro-architecture of the reduction cache** (Chapter **??**)
   - Introduction of a virtual memory partitioning scheme enabling multiple configurable regions.
   - Emphasis on flexibility and adaptability in cache design.
6. **Evaluation through RTL simulation** (Chapter **??**)
   - Assessment of the proposed cache architecture using RTL-level simulation.
   - Identification of two distinct groups of sparse matrices leading to different performance profiles.
7. **Future directions for performance and scalability** (Chapter **??**)
   - Proposal of a multicore version of the cache to extend scalability.
   - Exploration of bandwidth optimization strategies.

Chapter 1

Background

Contents

1.1 Introduction

MODERN computing systems are designed to deliver high performance by exploiting memory hierarchies, parallelism, and increasingly sophisticated hardware features. However, the efficiency of these systems is strongly tied to the regularity of memory access patterns. Applications with predictable, sequential data access can fully benefit from cache locality, prefetching mechanisms, and vectorization. In contrast, applications characterized by irregular accesses, where memory locations are not known in advance or follow non-contiguous patterns, are not able to achieve peak performance.

In domains such as graph analytics, sparse linear algebra, unstructured mesh computations, and database queries, the memory access patterns are often irregular. In these applications, pointer chasing, indirect indexing, and dynamically changing data structures disrupt spatial and temporal locality. As a result, the hardware struggles to hide the latency of memory accesses, leading to poor cache utilization and underutilization of compute ressources.

The impact of irregular accesses extends beyond raw performance. They also complicate algorithm design, hinder scalability on many-core architectures, and increase energy consumption due to repeated off-chip memory requests. Understanding and addressing these challenges is therefore a central concern in both computer design and application optimization. This chapter provides the necessary background for analyzing irregular accesses, highlighting their sources, effects, and the architectural considerations they expose, focusing on the case of indirect indexing in sparse matrix computation. The content of this chapter is partially based on the content from Papers I and II.

DEFINITION

Irregular accesses, or **"random" accesses**, are sequences of accesses where the sequence can not be easily predicted.

1.2 Sparse Matrices

SPARSE matrices are two dimensional arrays filled with many zeros and stored in compressed formats in memory. These sparse matrices are commonly used in diverse fields such as linear algebra, where sparse matrix multiplication is used to solve linear systems of equations for scientific purposes or simulation, as well as graph analytics where the vertices of a graph are represented as a sparse matrix [10], or finally transformers or other similar AI features where weights and activations can be represented and processed as sparse matrices [17, 18]. In these contexts, matrices are large, with dimensions up to billions of elements, and highly sparse, with non-zero densities down to 10^{-7} [33].

We note matrices with capital letters A, B, C, conversely to vectors a, b, c. Their dimension are denoted using M, N and K. For example, we can define a square matrix A of dimension M, or a matrix B of dimension (K, N). We note nnz the number of non-zero elements of a matrix, and d the associated density. With the example of a matrix $A(M, N)$:

$$d_A = \frac{nnz_A}{M \times N}$$

DEFINITION

A **sparse matrix** is a matrix filled with many zeros.

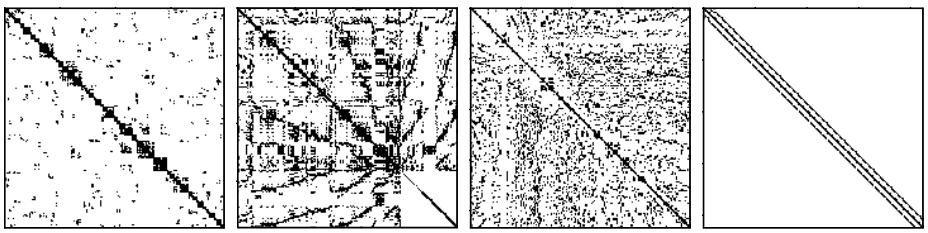
NOTE

In general, we consider a matrix to be highly sparse when its number of non-zero elements has the same order of magnitude than its dimension, $nnz \sim M$. But there is no intrinsic definition of the frontier between a sparse and a dense matrix. In this manuscript, we consider a matrix as sparse when it needs to be compressed in memory (see Section 1.3).

1.2.1 Banded Matrices

Matrices used in scientific computation originate from physical modelling where a system of partial differential equations is discretized, resulting in a large sparse matrix. In these systems, the forces between elements decreases quickly with their distance, creating banded matrices. Moreover, numerical techniques exist to transform matrices into banded matrices [36, 37, 51].

A banded matrix is a matrix which has a dense region around its main diagonal and which is sparser away from the diagonal. To give a few visual examples, Figure 1.1 shows the structure plots for four typical matrices found in the SuiteSparse Matrix Collection [33], having a high-density band along the diagonal. In Figure 1.2, we illustrate a banded, square matrix of dimension M. We define w_B as the width of the banded region, or *bandwidth*, as shown in *red* in the figure. Sometimes it is easier to refer to the half-*bandwidth*, w_{hB}, which respects the relation $w_B = 2 \times w_{hB} - 1$. The average density in the band and out of the band is defined as d_{IB} and d_{OB}, respectively. As in Figure 1.1d, in a 'perfectly' banded matrix, there are no elements outside the band, thus $d_{OB} = 0$.



(a) *roadNet-CA* **(b)** *scircuit* **(c)** *msc10848* **(d)** *raefsky3*

Figure 1.1: Examples of Banded Matrix Structure

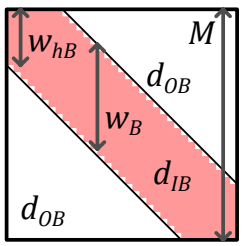


Figure 1.2: Banded Matrix Definition

DEFINITION

A **banded matrix** is a sparse matrix which has a higher density of non-zero elements around the diagonal. A **'perfectly' banded matrix** concentrates all its non-zero elements within a delimitated region around the diagonal. This region is called the band.

NOTE

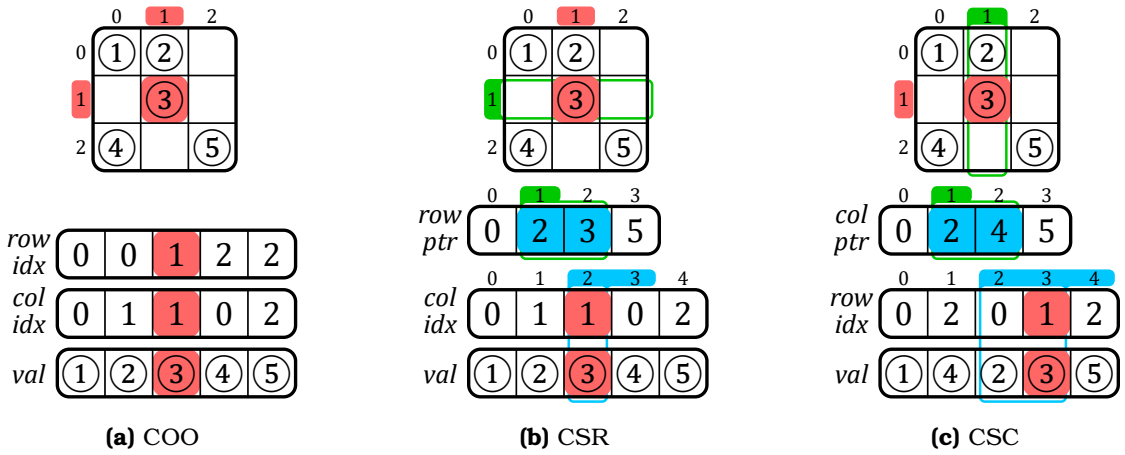
The SuiteSparse Matrix Collection [33] is a large, open source data base of sparse matrices from multiple real-world applications, many of which were provided by companies such as Boeing. It gathers around 3,000 matrices of various sizes, densities and structure, which are extensively used for benchmarking (see Chapter 2).

1.3 Sparse Formats

In the absence of a compressed format, matrices are stored in fixed-size two-dimensional arrays. With this dense format, elements can be randomly accessed ($\mathcal{O}(1)$), and sequential accesses are highly efficient. The two dimensions are called ranks which correspond to the rows and columns. For matrices, there are thus two variants of the dense format, depending on which rank is stored sequentially in memory. In a *row-wise* (or *row-major*) storage, the elements within the rows are sequential, and conversely for *column-wise* (or *column-major*).

Sparse matrices contain a very large number of zeros, and to avoid storing these repeated zeros, there exist many compressed formats [6, 7, 13, 30, 44, 48], where certain formats are best-suited for matrices with a specific structure. The main principle of these sparse formats is to avoid storing the zero elements, but this requires additional metadata which gives the position of the non-zero elements.

For example, let us consider the commonly used *COOrdinate* (COO) format that is relatively simple. It consists of three tables of size nnz with only the non-zero elements (val table in Figure 1.3a) and their row and column indices (row and col idx tables). The tuples do not necessarily need to be sorted, which makes it easy to append new entries. However, if the tuples are not sorted, this format is not well suited for sequential traversal or locating a specific matrix entry. This format can be sorted *row-wise* as in the example in Figure 1.3a or *column-wise* if needed, but in this case, it is less efficient than the two other widely used sparse formats *Compressed Sparse Row* (CSR) (Figure 1.3b) and its *column-major* counterpart *Compressed Sparse Column* (CSC) (Figure 1.3c), which are sorted by construction. With CSR, the val table of size nnz stores the non-zero elements sorted *row-wise* and the idx table of size nnz stores the corresponding column indices, similarly to COO. The third table ptr of size $M + 1$ stores the positions of the rows. For example, in Figure 1.3b, the second row of index 1 (*green*) has one non-zero element between positions 2 and 3 (*blue*, 3 excluded) of tables idx and val. The CSC format works similarly but in a *column-wise* manner: the val table of size nnz is sorted *column-wise* and the idx table of size nnz stores column indices. The ptr table of size $N + 1$ stores the positions of the columns. In the example in Figure 1.3c, the second column of index 1 (*green*) has non-zero elements between positions 2 and 4 (*blue*, 4 excluded) of tables idx and val. The ptr table allows CSR and CSC to have a lower memory footprint than COO.



(a) COO **(b)** CSR **(c)** CSC

Figure 1.3: Example of a Sparse Matrix in COO, CSR and CSC Formats

Many others classes of sparse formats exists such as *Blocked Compressed Sparse Row* (BCSR), *DIAgonal* (DIA), *ELLPACK* (ELL) and *HYBbrid* (HYB), which all have their own advantages depending on the matrix structure or the hardware setup. Additional information about these other formats is provided in Appendix A.

In general, there are many derivates from the above main categories of sparse formats. Ultimately, we can create any format we want depending on the software and hardware constraints. Later in this manuscript, we propose our own custom sparse format called *Banded Compressed Sparse Column* (BaCSC) that is an extension of CSC for banded matrices. As shown in Figure 1.4, we added extra information in the ptr table to identify the beginning and end of the band. Indeed, three pointers instead of one are now needed per column, as shown with the example of column 2 in *yellow*. The column is split in one in-band (IB) region in the middle and two out-of-band (OB) regions around. The ptr table size thus becomes $3 \times M + 1$. With this format, we can avoid the computational cost of identifying in which region non-zero elements are, while traversing the matrix, a feature we take advantage of in Chapter **??**.
(MOVE BaCSC in future chapter evaluation)

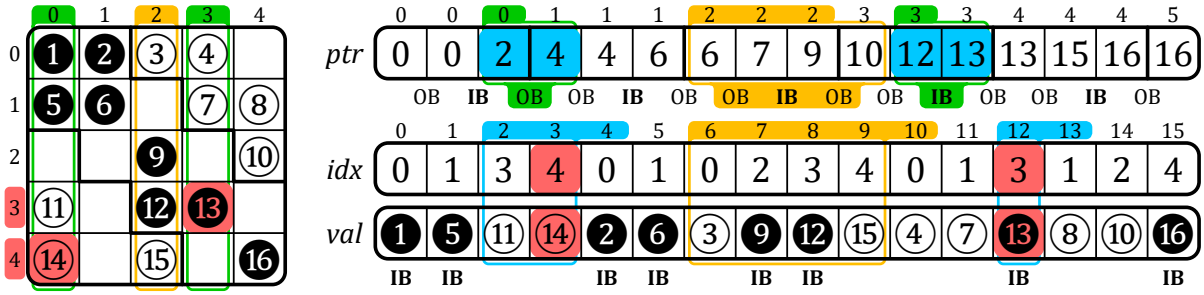


Figure 1.4: Example of a Sparse Matrix in BaCSC Format

DEFINITION

A **sparse format** is a memory representation of a sparse matrix which avoids storing zeros by using metadata.

NOTE

The most commonly used sparse formats are COO, CSR and CSC.

CONCLUSION

With the use of sparse formats, it is possible to greatly **decrease the memory footprint** of a sparse matrix while increasing the data dimensions. These formats **can be tailored** based on the needs of the software and hardware.

1.4 Algorithms for Dense and Sparse Matrix Multiplication

THE main matrix multiplication kernels that are of interest are Matrix-Vector (MV) and Matrix-Matrix (MM), which include SpMV and SpMSpV ($A \times b = c$), SpGEMV ($A \times b + d = c$), SpMM and SpMSpM ($A \times B = C$), and SpGEMM ($A \times B + D = C$), with 'Sp' for *sparse* and 'GE' for *general*. SpMV, as well as its variant SpGEMV, are by far the dominant operations in modern algebraic kernels (linear solvers, eigensolvers) which have been explicitly based on them [15, 20, 23, 28, 32, 41, 55]. Since these kernels are ubiquitous, their efficient implementation is primordial. In contrast, the SpMSpM multiplication is less frequent in standard algebraic kernels, but it is extensively used in algebraic graph manipulation [3, 9, 12, 14, 19, 29, 42], as well as in multigrid solvers [2, 8, 35, 43]. The growing importance of large graphs has motivated new research on optimizing SpMSpM applications.

In the following sections, we focus on SpMV and SpMSpM, with the notation $A(M,K) \times b(K) = c(M)$ and $A(M,K) \times B(K,N) = C(M,N)$, respectively. The rank K is the common rank between the two inputs A and B (or b). The existence of different ranks allows several algorithmic possibilities to compute matrix multiplication: the *Inner-Product* (`IP`) algorithm, the *Outer-Product* (`OP`) algorithm and *Gustavson*'s (`Gust`) algorithm [16, 22]. In all cases, the matrix A is traversed sequentially. The three algorithms, however, impose different access patterns for B (or b) and C (or c). Indeed, in many cases, they have to be accessed multiple times, as discussed in the following sections.

NOTE

In this manuscript, we focus on only two kernels, SpMV and SpMSpM. Note that, among MV kernels, SpMV is the most used and we specifically work on this kernel in Chapter **??**. SpMSpM is also widely used and studying this MM kernel is of interest as the memory access patterns in both matrices are irregular.
The *general* ('GE') version of these kernels is not fundamentally different, as the majority of the compute operations are in the matrix multiplication, so going forward, we will ignore the 'GE' variants.

1.4.0.1 Inner-Product Algorithm

The *Inner-Product* algorithm [16] is an intuitive way to calculate a matrix multiplication because it follows the loop order of the mathematical formulas (1.1) and (1.2) for MV and MM, respectively. The common rank K is the innermost loop of the algorithm (see Algorithms 1.1 and 1.2, lines 2 and 3, respectively). The output C (or c) is always updated sequentially, but the B-matrix (or b-vector) will be read entirely M times. In the context of sparse matrices, the reads of B (or b) are conditioned by the non-zero elements of A, which requires indirect accesses. If B (or b) is also sparse, the algorithm needs to perform *intersections* between the non-zero elements in the rows of A and the columns of B (or in the b-vector) (see Section 1.5). In Algorithm 1.2, exchanging M and N (lines 1 and 2) simply causes the output to be updated *column-wise*.

$$\forall i \in [\![0, M-1]\!],\ c_i = \sum_{k=0}^{K-1} A_{i,k} \times b_k \tag{1.1}$$

$$\forall (i,j) \in [\![0, M-1]\!] \times [\![0, N-1]\!],\ C_{i,j} = \sum_{k=0}^{K-1} A_{i,k} \times B_{k,j} \tag{1.2}$$

Algorithm 1.1: Dense Matrix-Vector *Inner-Product* Algorithm

```
1 for i ∈ [0, M − 1] do                    // row first
2 |   for k ∈ [0, K − 1] do
3 |   |   c_i += A_{i,k} × b_k
```

Algorithm 1.2: Dense Matrix-Matrix *Inner-Product* Algorithm

```
1 for i ∈ [0, M − 1] do                    // A-row first
2 |   for j ∈ [0, N − 1] do
3 |   |   for k ∈ [0, K − 1] do
4 |   |   |   C_{i,j} += A_{i,k} × B_{k,j}
```

1.4.0.2 Outer-Product Algorithm

The *Outer-Product* algorithm [16] has the opposite rank loop order than the *Inner-Product*. Indeed, Algorithms 1.3 and 1.4 show that the common rank K is the outermost loop (line 1). It changes nothing mathematically except that the B-matrix (or b-vector) is always read sequentially. In the context of sparse matrices, the output C (or c) is updated with indirect accesses. The algorithm needs to perform *reductions* of the partial sums $A_{j,k} \times B_{k,j}$ (or $A_{j,k} \times b_k$), as we show in Section 1.5. To avoid irregularity when updating C (or c), the **OP** algorithm is often separated into two phases. First, during the *multiply* phase, we compute several *partial outputs* (*POs*) consisting of intermediate matrices (or vectors) of partial sums $A_{j,k} \times B_{k,j}$ (or $A_{j,k} \times b_k$), then during the *merge* phase, we accumulate all the *POs* to form the final C-matrix (or c-vector). With this method, **OP** necessitates the generation of at most K *POs* of densities conditioned by the non-zeros elements of both inputs A and B (or b).

Algorithm 1.3: Dense Matrix-Vector *Outer-Product* Algorithm

```
1 for k ∈ [0, K − 1] do                    // column first
2 |   for i ∈ [0, M − 1] do
3 |   |   c_i += A_{i,k} × b_k
```

Algorithm 1.4: Dense Matrix-Matrix *Outer-Product* Algorithm

```
1 for k ∈ [0, K − 1] do // common rank first
2 |   for i ∈ [0, M − 1] do
3 |   |   for j ∈ [0, N − 1] do
4 |   |   |   C_{i,j} += A_{i,k} × B_{k,j}
```

1.4.0.3 Gustavson's Algorithm

As the MM algorithms have three ranks instead of two for MV, there is a third loop ordering, known as *Gustavson*'s algorithm [22] where the common rank K is the middle-loop (see line 2 in Algorithm 1.5). With this approach, the output is computed row-by-row with *PO*-rows generated and accumulated for each output C-row. With this algorithm, updates of the C-rows are sequential, but elements within the rows are updated with indirect accesses. In the dense case, the B-matrix is entirely read M times and K *PO*-rows, of size N, are generated. In the context of sparse matrices, conversely to the C-rows, the selection of the rows in B is indirect, but the accesses are sequential within a row. Thus, the *intersection* search is only needed for the B-rows and not each element within rows. The *PO*-rows can be accumulated before computing the next C-row. Overall, this algorithm can be seen as a trade-off between `IP` and `OP` for MM kernels, since the indirect accesses are shared between B and C. Again, for this algorithm, exchanging M and N causes the three matrices to be accessed *column-wise* instead of *row-wise*.

Algorithm 1.5: Dense Matrix-Matrix *Gustavson*'s Algorithm

```
1 for i ∈ [0, M − 1] do
2 |   for k ∈ [0, K − 1] do    // common rank
3 |   |   for j ∈ [0, N − 1] do
4 |   |   |   C_{i,j} += A_{i,k} × B_{k,j}
```

1.4.0.4 Comparison of the Algorithms

In Table 1.1, we summarize the main features of `IP`, `OP` and `Gust` in terms of accesses to the matrix A, B (or b) and C (or c). From this high level analysis, we can easily see that the data that is accessed with indirections shifts depending on the algorithm, from the input B (or b) to the output C (or c), with `Gust` being a trade-off between `IP` and `OP` for MM multiplication. To have a better understanding of the consequences on the actual number of memory accesses, we propose a more detailed analysis in Section 1.5.4, for the case of SpMSpM.

NOTE

Note that these algorithms are available in dense as well as in sparse MV and MM multiplications. In the sparse variants, the above algorithms (from 1.1 to 1.5) have the same behavior from a mathematical point-of-view, but need adjustments to traverse a sparse format instead of a dense matrix. Sparse versions of these algorithms are presented in Section 1.5.

CONCLUSION

There are *three main algorithms* to compute sparse MV and MM kernels, ***Inner-Product*** **(`IP`)**, ***Outer-Product*** **(`OP`)** and ***Gustavson*** **(`Gust`)**. Each presents different characteristics, the main one being the **irregularity of the accesses**: on the **input** matrix B (or vector b) with `IP`; on the **output** matrix C (or vector c) with `OP`; on both with `Gust` but at **row scale**.

1.4.1 Tiling

The *tiling* process [21] consists in partitioning a matrix: a matrix may be divided into *tiles*, non-overlapping sub-matrices. In other words, this method consists of adding ranks to delimit the *tiles*, and thus adding loop iterations in the kernel algorithm.

Tiling methods make it possible to process smaller portions of the matrix and thus to adapt the working set to the size of the local memory [34, 52], particularly when the matrix has a structure with denser regions. *Tiling* can also increase parallelism opportunities. Software optimizations for sparse matrix computation, such as OSKI [49] and Sparse BLAS [15], are based on *tiling*. In previous sections, the study of the `IP`, `OP` and `Gust` algorithms highlighted the influence of rank ordering on the sequentiality and data movement, and thus, adding intermediate ranks with *tiling* breaks the regularity of pure `IP` or `OP` algorithms, requiring additional hardware support (see Section 1.5).

For example, let us consider the A-matrix in Figure 1.5 which is divided into four *tiles* (one level of *tiling*). For a SpMV, two new ranks appear for a total of four ranks, as shown in

Table 1.1: Comparison of Matrix Multiplication Algorithms

	***Inner-Product* (IP)**[1]	***Outer-Product* (OP)**[1]	***Gustavson* (Gust)**[2]
A(M,K) **Accesses**	• *row-wise* • read N times[3,4] • sequential	• *column-wise* • read once • sequential	• *row-wise* • read once • sequential
B(K,N) **Accesses**	• *column-wise* • read M times[5] • not sequential	• *row-wise*[6] • read $M \times d(A)$ times[4,7] • sequential	• *row-wise* • read $M \times d(A)$ times[7] • sequential within rows
C(M,N) **Accesses**	• generated *row-wise*[6] • one sequential traversal	• generated without specific structure • K *POs* produced[8] (each *PO* has $\overline{nnz_c}(A) \times \overline{nnz_r}(B)$ non-zero elements[9])	• generated *row-wise* • $K \times d(A)$ *PO*-rows produced[10] for each row of A (M) ($\overline{nnz_r}(B)$ non-zero elements per *PO*-row)

[1] Considering $N = 1$ in the case of MV multiplication.
[2] Only for MM multiplication.
[3] Or less than N times if B has empty columns.
[4] Read once in the case of MV multiplication.
[5] If B is stored with a sorted sparse format, else it would be read $\overline{nnz_r}(A) \times M = nnz(A)$ times.
[6] *element-wise* in the case of MV multiplication (there is no row).
[7] Which is equal to $\overline{nnz_c}(A)$, the average number of non-zero elements in the columns of A.
[8] Or less than K times if A has empty columns or B has empty rows.
[9] $\overline{nnz_c}(A)$ in the case of MV multiplication.
[10] Which is equal to $\overline{nnz_r}(A)$, the average number of non-zero elements in the rows of A.

Algorithm 1.6: M_1 and K_1 allow *tile* selection and M_2 and K_2 allow data access within a *tile*. Thus, we could decide to apply an `OP` on *tiles* and conversely an `IP` within *tiles*, mixing-up the pros and cons of these two algorithms at different rank levels. In the case of the example in Figure 1.5, the upper-left A-*tile* (*red*) is evaluated first, sequentially updating the upper *tile* of the c-vector in an `IP` manner. The lower-left A-*tile* (*yellow*) is then computed the same way. At this point, *intersections* have been done with only the upper b-*tile*, and c has been updated sequentially (first *PO*). The two next A-*tiles* (*green*, then *blue*) will compute *intersections* with the lower b-*tile* and update the entire c-vector sequentially, showing the `OP` characteristic of having *POs* to *merge*, two in this case.

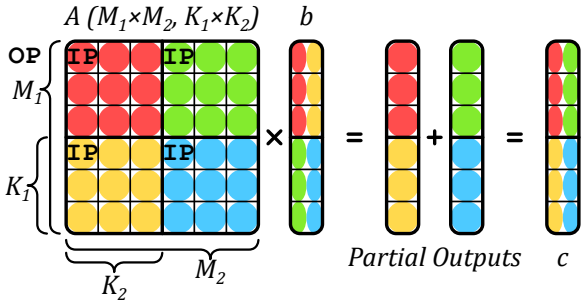


Figure 1.5: An Example of *Tiling* on SpMV

Algorithm 1.6: Matrix-Vector Algorithm with Custom *Tiling* of Figure 1.5

```
1 for k1 ∈ [0, K1 − 1] do
2     for i1 ∈ [0, M1 − 1] do
3         for i2 ∈ [0, M2 − 1] do
4             for k2 ∈ [0, K2 − 1] do
5                 i ← i1 × M2 + i2
6                 k ← k1 × K2 + k2
7                 ci += Ai,k × bk
```

DEFINITION

Tiling is a method for partitioning matrices in non-overlapping sub-matrices, called ***tiles***.

NOTE

Note that in case of *tiling*, the A-matrix is no longer read sequentially, which can be problematic if it is stored in a classical sparse format like CSR that is specifically made for *row-wise* accesses. Custom sparse formats adapted for *tiling* can be used [25], but this requires preprocessing to perform the conversion.

CONCLUSION

Tiling methods are widely used to **distribute workloads** across multicore systems, and to **locally reduce the data dimensions**. Complex *tiling* algorithms with many possible rank ordering **inherit the behaviors of `IP` and `OP`**, and can be described as a combinations of both at different *tile* levels.

1.5 Hardware Challenges for Sparse Matrix Computation

In the previous section we presented a brief background on matrix multiplication without considering the underlying hardware. Although the various sparse formats reduce the memory footprint, they have the disadvantage that indirect accesses are required, which creates data-dependencies and reduces the effectiveness of caches. As we have seen, with all three algorithms, at least some of the accesses are irregular. The lack of spatial locality for these accesses reduces the benefit of having caches [38, 54]. In other words, a full cache-line is loaded, but only a few entries are read or modified. Moreover, the matrices of interest are extremely large and even the non-zero elements, and associated metadata, can not generally fit in the cache. Depending on the algorithm, successive accesses to the same element may be too far apart to benefit from temporal locality in the cache.

1.5.1 Inner-Product Algorithm

In an `IP` algorithm, the A-metadata needs to be read to indirectly access the B-matrix (or b-vector), which may itself be stored in a sparse format. In this case, the algorithm needs to check if the non-zero A-indices are present in B (or b). If an index appears in neither list, there is no need to perform the multiplication (ineffectual operation). More generally, this problem applies to any *fiber* and is called the *intersection* search. As a minimum, this requires reading all the metadata for A and B (or b), and identifying those present in both.

In Algorithm 1.7, we present the pseudo-code for an *Inner-Product* (`IP`) SpMSpM with A and B stored in CSR and CSC formats, respectively. The outer-loop (M, line 1) iterates over the rows of A. The middle-loop (N, line 2) iterates over the columns of B. The inner-loop (line 4) iterates over the non-zero elements of the current row (i) of A, and for each index (k_A of position pos_A in the idx array of A, line 5), a search is performed in the metadata of B (line 6), to see if the corresponding index in B is present. Assuming it is ($isMatch$ is true, line 7), the partial sum is computed and then accumulated locally (acc, line 8). This example highlights the need to efficiently search for the *intersection* of indices in the metadata of the inputs.

Different algorithms can be used to address the *intersection* search, depending on whether the elements are sorted. When the elements are sorted, the *intersection* becomes a *merge* and generally requires at most $2 \times nnz - 1$ comparisons (in some cases the number of comparisons can be reduced [5]). In Algorithm 1.7, we use sorted sparse formats (CSR and CSC). Thus, the naïve *intersection* search proposed (line 10) does at most one traversal of each the current row of A (i) and the current column of B (j). Note that computing an *intersection* in software induces the use of conditional loops and branches, which can be costly for classical CPU branch predictors. In case the matrix B (or vector b) is stored in a dense format, the *intersection* search is simplified, since all indices are implicitly present, but the accesses to B (or b) remain indirect. As we see in Chapter 2, certain accelerators provide support for performing the *intersection* operation directly in hardware.

DEFINITION

The ***intersection* search** consists in founding non-zero elements of two sparse arrays that have matching indices. This search is costly in software on classical CPUs, thus being key to improve performance of `IP` algorithm.

1.5.2 Outer-Product Algorithm

In an `OP` algorithm, the inputs A and B (or b) are accessed sequentially, but the updates of the output C (or c) require indirect accesses. The pattern of the accesses, while the output is generated, is irregular and determined by the structure of the inputs. Furthermore, if these "random" updates provoke cache misses where only one word within a cache-line is modified, the bandwidth to external memory is poorly utilized. The `OP` algorithm generates several *partial outputs* (*POs*) that must be accumulated to obtain the final output. We can

Algorithm 1.7: `IP` SpMSpM Performing *Intersection* Searches with CSR and CSC Formats

Data: $A(M, K)$ in CSR: $A.val, A.idx, A.ptr$
Data: $B(K, N)$ in CSC: $B.val, B.idx, B.ptr$
Data: $C(M, N)$ in dense format (for readability)
Result: $C = A \times B$

```
1  for i ∈ [0, M − 1] do                                   // Iterations over the rows of A
2      for j ∈ [0, N − 1] do                               // Iterations over the columns of B
3          acc ← 0; pos_B ← B.ptr_j;
4          for pos_A ∈ [A.ptr_i, A.ptr_{i+1} − 1] do
               // Iterations over the non-zero elements of the i-th row of A
5              k_A ← A.idx_{pos_A};
               // Column-index of the (i, pos_A) non-zero element of A
6              (isMatch, pos_B) ← IntersectionSearch(k_A, j, pos_B, B.idx, B.ptr);
               // Return true and the matching pos_B if idx exists in the j-th column of B
7              if isMatch then                              // If indices did match
8                  acc ← acc + A.val_{pos_A} × B.val_{pos_B};
                   // Local partial sum accumulation
9          C_{i,j} ← acc;                  // Write in memory of the resulting (i, j) element of C
10 function IntersectionSearch(k_A, j, pos_B, B.idx, B.ptr) is
       // Naive intersection search allowing the i-th row of A and the j-th column of B to
          be traversed only once at iteration (i, j)
11     while B.idx_{pos_B} < k_A and pos_B < B.ptr_{j+1} do
12         pos_B ← pos_B + 1;
13     if B.idx_{pos_B} = k_A then                  // Returns true if indices from A and B match
14         return (true, pos_B);
15     return (false, pos_B);
```

identify two strategies: *immediate update* and *deferred update*. In the former, the final output is immediately updated as each partial sum is generated. If the output is stored using a sparse format, either the index being updated does not yet exist, and the new partial sum must be inserted. Or, if it exists, a lookup must be done to locate it, the update performed and the result written back. The problem associated with combining the *POs* is called *reduction*.

In Algorithm 1.8, we present two pseudo-code for an *Outer-Product* (`OP`) SpMSpM using *immediate update* (from line 1) and *deferred update* (from line 7) with A and B stored in CSC and CSR formats, respectively. The outer-loop (K, line 1 or 7) iterates over the columns of A and the rows of B (common rank). The middle-loop (line 2 or 9) iterates over the non-zero elements of the current column (k) of A. The inner-loop (line 4 or 11) iterates over the non-zero elements of the current row (k) of B. Because we only iterate over the non-zero values, there is no need for an *intersection* search. The partial sum is then computed (line 6 or 15). Finally, in the *immediate update* version, the partial sum must be accumulated to C (line 6). These accesses are irregular, making it necessary to lookup the value in C, update it, and write it back. This is a *reduction* operation, performed on irregular data.

To avoid the complexities of these "random" *reductions* with *immediate updates*, the *deferred update* version postpones the accumulation and simply stores the partial sums in memory. One such technique is called *output batching* [31]. With this technique, arrays containing the indices and the partial sum are streamed sequentially to memory (PO_k, lines 13-15). Then, during a second pass (line 17), they are read back and the accumulations are performed, an approach which results in sequential memory accesses during both the first and second passes, with the downside that the volume of data transferred to memory is increased. As we see in Chapter 2, hardware accelerators may combine the *immediate* and *deferred update* techniques to optimize memory bandwidth.

DEFINITION

"Random" *reductions* occur when updating data from memory at irregular addresses, which happen on *immediate update* `OP`. With *deferred update* `OP`, the *reductions* are sequential but large sets of data need to be buffered in memory. Optimizing *reduction* operations is key to improving the performance on `OP` algorithm.

Algorithm 1.8: OP SpMSpM Performing *Reduction* with CSR and CSC Formats

Data: $A(M, K)$ in CSR: $A.val, A.idx, A.ptr$
Data: $B(K, N)$ in CSC: $B.val, B.idx, B.ptr$
Data: $PO_k(M, N)$ for all $k \in [\![0, K-1]\!]$ in COO: $PO_k.val, PO_k.row, PO_k.col$
Data: $C(M, N)$ in dense format (for readability)
Result: $C = A \times B$

```
// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * *
// * * * 'Immediate updates' version * * * * * * * * * * * * * * * * * * * * * * *
1  for k ∈ [[0, K−1]] do                          // Iterations over the columns of A and the rows of B
2      for pos_A ∈ [[A.ptr_k, A.ptr_{k+1}]] do
           // Iterations over the non-zero elements of the k-th column of A
3          i_A ← A.idx_{pos_A};                   // Row-index of the (pos_A, k) non-zero element of A
4          for pos_B ∈ [[B.ptr_k, B.ptr_{k+1}]] do
               // Iterations over the non-zero elements of the k-th row of B
5              j_B ← B.idx_{pos_B};
               // Column-index of the (k, pos_B) non-zero element of B
6              C_{i_A,j_B} ← C_{i_A,j_B} + A.val_{pos_A} × B.val_{pos_B};
               // Reduction (to memory) of the partial sum into the (i_A, j_B) element of C
                  (immediate update)
// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * *
// * * * 'Deferred updates' version  * * * * * * * * * * * * * * * * * * * * * * *
// Multiply phase
7  for k ∈ [[0, K−1]] do                          // Iterations over the columns of A and the rows of B
8      pos_PO ← 0;
9      for pos_A ∈ [[A.ptr_k, A.ptr_{k+1}]] do
           // Iterations over the non-zero elements of the k-th column of A
10         i_A ← A.idx_{pos_A};                   // Row-index of the (pos_A, k) non-zero element of A
11         for pos_B ∈ [[B.ptr_k, B.ptr_{k+1}]] do
               // Iterations over the non-zero elements of the k-th row of B
12             j_B ← B.idx_{pos_B};
               // Column-index of the (k, pos_B) non-zero element of B
13             PO_k.row_{pos_PO} ← i_A;
14             PO_k.col_{pos_PO} ← j_B;
15             PO_k.val_{pos_PO} ← A.val_{pos_A} × B.val_{pos_B};
               // The k-th partial sum of the (i_A, j_B) element of C is stored in memory in the
                  k-th partial output
16             pos_PO ← pos_PO + 1;
       // Each partial output is generated sorted, row-wise
// Merge phase
17 for k ∈ [[0, K−2]] do                          // Naive serial merge of the K partial outputs
18     PO_{k+1} ← 2DListsMerge(PO_k, PO_{k+1});
// PO_{K-1} is C stored in COO format
19 for pos ∈ [[0, length(PO_{K−1}) − 1]] do
       // Update C (dense) from the fully merged partial outputs
20     i ← PO_{K−1}.row_{pos};
21     j ← PO_{K−1}.col_{pos};
22     C_{i,j} ← PO_{K−1}.val_{pos};                // Write in memory of the (i, j) element of C
```

1.5.3 Summary

In Table 1.2, we summarize the advantages and hardware challenges for each algorithm, including **Gust** which presents a trade-off between **IP** and **OP**. The potential for parallelism and data-reuse varies across algorithms, **IP** being the most limited unless *tiling* methods are used. With **OP**, *POs* can easily be generated across a multicore system but at the cost of transferring a large volume of data to memory. With *immediate updates*, **OP** transfers less data but all accesses on the output are "random" *reductions*.

Table 1.2: Comparison of Advantages and Challenges with Matrix Multiplication Algorithms

		***Inner-Product* (IP)**	***Outer-Product* (OP)**	***Gustavson* (Gust)**
Access Count	**MV**	***A*:** once ***b*:** $nnz(A)$ ($\sim M\times$) ***c*:** once	***A*:** once ***b*:** once ***c*:** $nnz(A)$ ($\sim K\times$)	*N/A*
	MM	***A*:** $N\times$ ***B*:** $M\times$ ***C*:** once	***A*:** once ***B*:** $M \times d(A)\times$ ***C*:** $nnz(A) \times \overline{nnz_r}(B)$ ($\sim K\times$)	***A*:** once ***B*:** $M \times d(A)\times$ ***C*:** $nnz(A) \times \overline{nnz_r}(B)$
Sequential Accesses		Accesses to A and C	Accesses to A and B	Accesses to A and C Accesses to B-rows
Input Format		Accesses to A and B benefit from alternate formats (CSR and CSC)		Allows consistent format for A and B
Parallelism Opportunity		Possible with *tiling*	Generation of *POs* over common rank of A and B	Reading A-rows and updating C-rows
Reuse Opportunity		Potentially B, but limited by its size	An A-column or B-row while computing *POs*	A set of B-rows, based on non-zero elements of A-rows
Challenges		*Intersection* search between non-zero elements in A and B	Storage for *POs* or in-place *merge* and *reduce*	*Merge* and *reduce* of *PO*-rows

The issues presented with **IP** and **OP** can be generalized to two concepts: *gather* and *scatter*. The *gather* problem exists when the inputs are indirectly accessed, and need to be read from memory using metadata to present the processor with the correct terms for computing the partial sums. Typically, with **IP**, the *gather* problem is more difficult. The *scatter* problem exists when the order in which the output matrix/vector is generated is not sequential and is typically associated with the **OP** algorithm. In the case of *tiling* or with **Gust**, both *gather* and *scatter* must be performed creating a need for efficient *intersection* searches and random *reductions*.

CONCLUSION

There are **two main challenges** to address when computing sparse matrices. When reading irregular data from memory, addressing the ***gather*** problem is key. It consists of processing efficient ***intersection* searches**. When updating irregular data from memory, addressing the ***scatter*** problem is key. It consists of **managing *POs*** in memory or processing efficient **"random" *reductions***. These two challenges are tied to **IP** an **OP**, respectively, and are present in any higher rank algorithms for sparse matrix computing.

1.5.4 SpMSpM Algorithm Analysis

To better understand how the algorithms impact the number and types of memory accesses, we developed, and presented in Figure 1.6, a model based on memory accesses counts for SpMSpM. We identify five memory access patterns: (1) those where a matrix is read only once, thus there is no opportunity for reuse, (2) where the accesses are sequential, but a *fiber* is read multiple times, (3) where *fibers* are read consecutively, but the accesses within a *fiber* are "random", (4) where the *fibers* are accessed "randomly", but within a *fiber* the accesses are sequential, and (5) where the matrix elements are accessed multiple times, and with a "random" access pattern.

For each of the three algorithms, we counted the number of read and write accesses, based on the density of the input matrices, and the local memory storage available ("0D", "1D" and "2D"), as presented in Table 1.3. We refer to a system with no local storage or cache as '0

Dimensional' ("0D"), a system which has local storage that is sufficient to hold a single row or column as '1 Dimensional' ("1D"), and a system which can cache an entire matrix as '2 Dimensional' ("2D"). In Figure 1.6, we plotted the number of memory accesses for SpMSpM, in a logarithmic scale, based on a square matrix with $M = 10,000$. Indeed, the number of non-zero elements in C depends on the structure of the input matrices and in this model, we have assumed the non-zero entries in the input matrices are uniformly, randomly distributed. This corresponds to a *worst case* for sparse matrices (no spatial and temporal locality). If A or B are banded matrices, for example, there would be fewer non-zero elements in C. In each graph, we have separated the accesses into A (*bottom*), B (*middle*) and C (*top*). The color code shows the access pattern, based on the five categories described above.

Table 1.3: Memory Access Count for Each Matrices of **IP**, **OP** and **Gust** SpMSpM

	***Inner-Product* (IP)**	***Outer-Product* (OP)**	***Gustavson* (Gust)**
$\mathbf{A}(M,K)$	**0D**: $N \times nnz(A)$ **1D-2D**: $nnz(A)$	**0D-1D-2D**: $nnz(A)$	**0D-1D-2D**: $nnz(A)$
$\mathbf{B}(K,N)$	**0D-1D**: $M \times nnz(B)$ **2D**: $nnz(B)$	**0D**: $M \times d(A) \times nnz(B)$ **1D-2D**: $nnz(B)$	**0D-1D**: $M \times d(A) \times nnz(B)$ **2D**: $nnz(B)$
$\mathbf{C}(M,N)$	**0D-1D-2D**: $nnz(C)$ (max: $M \times N$)[1]	**0D-1D**: $2 \times M \times d(A) \times nnz(B)$ **2D**: $nnz(C)$ (max: $M \times N$)[1]	**0D**: $2 \times M \times d(A) \times nnz(B)$ **1D-2D**: $nnz(C)$ (max: $M \times N$)[1]

[1] We can not easily quantify $nnz(C)$ knowing $nnz(A)$ and $nnz(B)$. We thus use a statistic approximation.

In the top-row of Figure 1.6 (graphs ①, ② and ③), we start by considering a naïve system which has no local storage ("0D"), thus all data comes from main memory. From the graphs in this row, we clearly see that **IP** requires more accesses at low density, as the A-matrix must be read N times (graph ①). Whereas for **OP** and **Gust**, the A-matrix is read only once (thus the *grey* color). For **OP** and **Gust** the total number of accesses is the same, however, with **OP** (graph ②), the accesses to C are "random" (*red*). With **Gust** (graph ③), the accesses to B are sequential within the rows (*orange*) and the C-rows are generated sequentially (*yellow*), thus **Gust** avoids having any fully "random" accesses (*red*).

Since main memory accesses are the principal bottleneck, systems integrate local storage (such as buffers or caches). Thus, we consider the case of a system with sufficient local memory to store one full row, per matrix (equivalent to few sparse rows, "1D"). We assume this local storage is perfect (no misses) and we re-calculate the number of remaining main memory accesses, as shown in the middle-row of Figure 1.6 (graphs ④, ⑤ and ⑥). The matrices are assumed to be initially stored in main memory, thus they must be accessed at least once. As can be seen, for **IP** (graph ④), the number of A-accesses is reduced, because rows are reused, thus the total number of A-accesses becomes equal to that in **OP** and **Gust**. For **OP** (graph ⑤), the single row buffer greatly reduces the number of B-accesses, however, it does nothing for C (despite the appearance, due to the logarithmic scale). For **Gust** (graph ⑥), the single row buffer is sufficient to cache the *reduction* operations in C, and thus, it now has fewer overall accesses than **OP**. It is also interesting to note that **OP** and **Gust** scale better than **IP** with lower densities, as seen by the shallower slope.

Finally, we have considered the extreme case where an system has a buffer of sufficient size to store two sparse matrices ("2D"), which is shown in the bottom-row of Figure 1.6 (graphs ⑦, ⑧ and ⑨). Although this case is not practical for large matrices, through the proper use of *tiling* to locally reduce the dimensions of a sub-matrix, this case can be achieved at the level of a *tile*. In this case, the total number of external accesses is equal for the three algorithms, as each matrix is accessed only once from main memory. For **IP** (graph ⑦), it is interesting to note that such a large buffer is needed to locally address the *intersection* search in B. Similarly for **OP** (graph ⑧), a large storage is required to address the *reductions* in C. For **Gust**, in fact, it is not necessary to buffer the entire B-matrix. Indeed, buffering a few rows (corresponding to $nnz_r(A)$) is sufficient to achieve the number of accesses shown in graph ⑨. This corresponds to an intermediate design point (multiple row buffers for B) not explicitly shown in the figure (between graphs ⑥ and ⑨).

When viewed overall, Figure 1.6 highlights the benefits of **Gust**. We clearly see that **Gust** does not require any fully "random" accesses (*red*). With a single row buffer, it is possible to address the *reductions* in C. And, with a limited number of row buffers, the number of external memory accesses for B can be reduced. Based on this analysis, it is clear that, among the three base algorithms, **Gust** is the best candidate for hardware implementation of

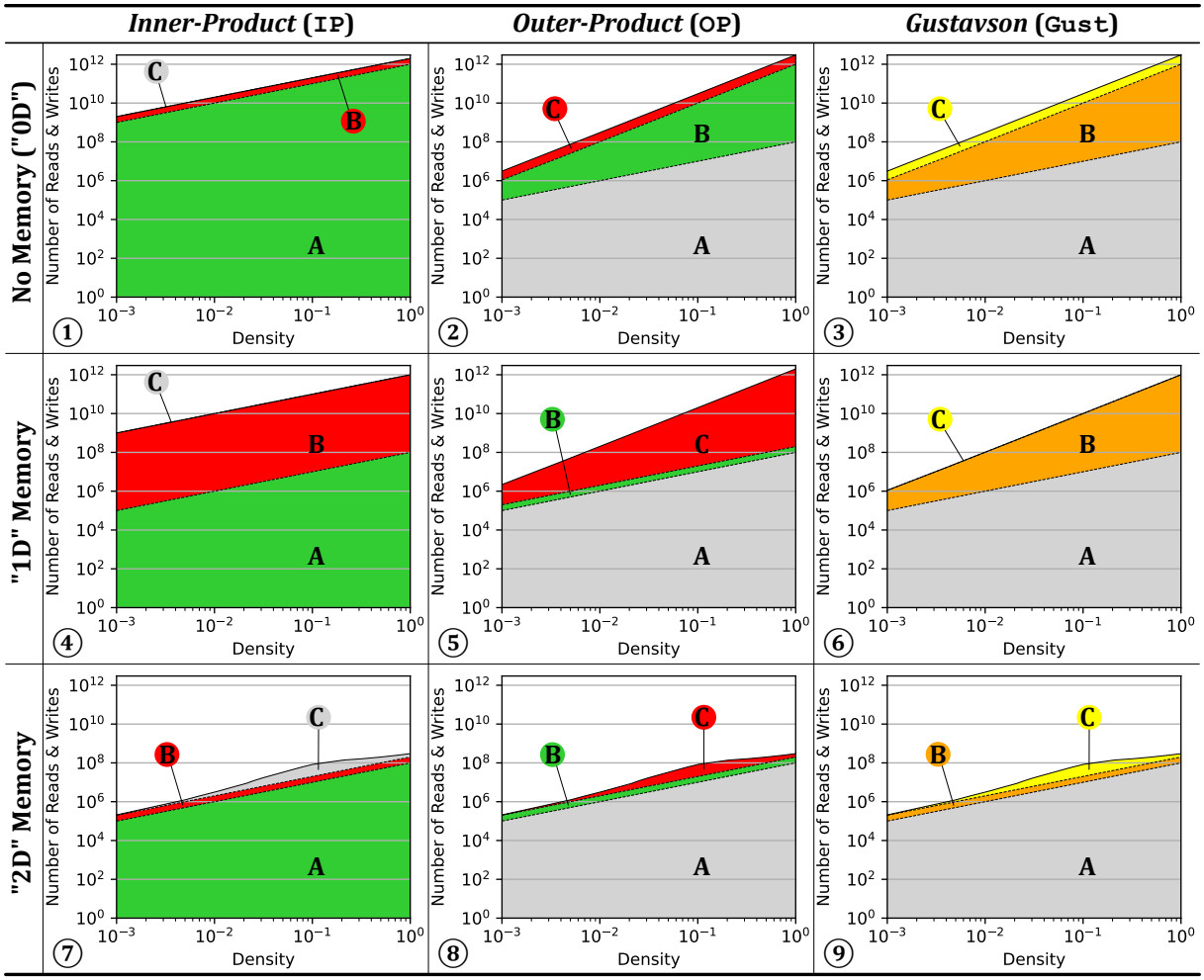


Figure 1.6: Model of Memory Access Counts Depending on Algorithm and Local Memory Size

SpMSpM.

NOTE

For SpMV, such an analysis does not show significant differences between `IP` and `OP`, as both algorithms require similar number of memory accesses. However, `OP` shows more potential for parallelism and data-reuse as seen in the previous section (1.5.3), thus being preferred for SpMV hardware accelerators.

CONCLUSION

Based on a high level analysis of the memory accesses on SpMSpM, ***Gustavson*'s algorithm** appears as the **best trade-off** to minimize memory traffic with reasonable amount of local storage. The second best option is `OP`, which requires fewer memory accesses than `IP` when the matrices have a low density.

1.6 Conclusion

In this chapter, we established the foundation for understanding the relationship between applications working with irregular data and the underlying challenges in hardware. Irregular accesses are common in sparse computing and cause reduced cache efficiency, limited parallelism, and higher memory latency exposure. We analysed the most representative algorithms for sparse matrix multiplication kernels, and identified the operation that provoke irregular accesses. *Intersections* and *reductions* on irregular data with classical systems are costly, thus they represent the central challenge for efficient computation on sparse data. To improve the performance of these operations, specialized hardware has been proposed as will be seen in the next chapter.

TAKEAWAYS

Sparse matrices: large, highly sparse, structured (such as banded matrices)
Sparse formats: diverse, reduce memory footprint
- COO, the simplest
- CSR and CSC, the mostly used
- BaCSC, our own sparse format for banded matrices

Sparse kernels: sparse MV and MM multiplications (such as SpMV and SpMSpM), generate indirect accesses
Algorithms: *Inner-Product* (`IP`), *Outer-Product* (`OP`) and *Gustavson* (`Gust`), extended with *tiling* methods
- Best potential to reduce memory transfers: `Gust`, followed by `OP` at low density

Problem: irregular accesses, generate high memory transaction latencies
Challenges:
- *gather*: *intersection* searches
- *scatter*: "random" *reductions*, or storing and *merging partial outputs* (*POs*)

Chapter 2

State-of-the-Art

Contents

2.1 Introduction

TODO

Chapter 3

High Level Architectural Presentation of SpDCache

Contents

3.1 Introduction

In the previous chapters, we analyzed the causes and effects of irregular accesses in sparse computations and identified the *Outer-Product* (`OP`) algorithm of Sparse Matrix-Vector multiplication (SpMV) ($A \times b = c$) as a key case. The `OP` algorithm generates irregular updates on the output vector c, as each non-zero element of the input matrix A contributes to distinct, and often distant, memory locations. These "random" *reductions* are a major challenge for cache-based architectures.

Conventional cache hierarchies are designed for regular, predictable access streams that exploit spatial and temporal locality. In contrast, `OP` SpMV exhibits poor locality, leading to low cache utilization, frequent evictions, and inefficient bandwidth use.

Previous works, such as PHI [39], SortCache [45], and ASA [56], have proposed *reduction*-aware cache mechanisms to alleviate irregularity by improving accumulation and storage efficiency. However, these solutions do not exploit the structural properties of the processed matrices, and therefore miss optimization opportunities when partial regularity is present. This observation motivates our focus on banded sparse matrices, which combine overall sparsity with localized dense regions near the diagonal. Such matrices provide a controlled environment to study how structure and locality can be leveraged to enhance cache performance.

This chapter begins by recalling the main concepts of generic data-caches, introducing the terminology and mechanisms required to discuss caching behavior under irregular access conditions. It then presents a *reduction*-aware cache architecture, designed to efficiently support "random" *reductions* in `OP` SpMV. The proposed design, SpDCache (presented in Paper II), extends traditional caching models with region-based organization and native *reduction* support. Finally, we evaluate its performance through high-level modeling on banded and non-banded matrices, highlighting the influence of data regularity on cache efficiency.

3.2 Generic Data-Caches

MODERN processors and accelerators integrate data-caches to bridge the latency and bandwidth gap between computation units and main memory. A cache is a small, fast memory that stores recently accessed data to anticipate future reuse. Its efficiency relies on two main forms of locality:

- temporal locality, where recently accessed data is likely to be reused soon, and
- spatial locality, where data stored near recently accessed addresses is likely to be used next.

By exploiting these properties, caches reduce the average access latency and increase the overall bandwidth efficiency.

A cache is composed of several cache-lines, each grouping contiguous memory words. The cache-line is the minimum granularity of data movement between memory levels. When an access occurs, the cache controller searches for the requested address among the currently stored cache-lines. If the address is found, a hit occurs and the data is served directly to the core. Otherwise, a miss occurs, triggering a memory fetch from a lower cache level or from main memory. If necessary, an existing cache-line is evicted to make room for the new data. Each cache-line is associated with *tags* and validity bits that record its address and coherence state.

Several mapping policies exist. In a direct-mapped cache, each memory block maps to a single cache-line, which is simple but prone to conflicts. A n-*way set*-associative cache allows each block to be placed in one of n possible cache-lines within a *set*, improving hit rate at moderate cost. A *fully associative* cache removes these constraints, as any block may occupy any cache-line, but requires more complex *tag* searches. When multiple candidates exist, a replacement policy such as *Least Recently Used* (*LRU*) or *Random* decides which entry is evicted.

Write policies also impact performance. With a *write-through* policy, every store operation updates both the cache and main memory, maintaining consistency but increasing traffic. A *write-back* policy delays the update until eviction, reducing bandwidth usage at the cost of additional state bits (dirty flags) to maintain consistency. For workloads dominated by sequential accesses, these mechanisms are highly effective. However, irregular accesses tend to degrade cache efficiency by increasing miss rates and provoking frequent evictions.

Modern processors implement hierarchical cache organizations to balance latency, capacity, and throughput. A typical CPU integrates small L1 caches close to each core, larger L2 caches, and a high-capacity L3 cache shared at the chip level. Accelerators and domain-specific architectures often employ local data storages or scratchpad memories serving the same purpose, providing low-latency access to frequently reused data (see Chapter 2).

The generic cache assumes regular read and write operations with predictable locality. Yet workloads operating on sparse or *reduction*-intensive data violate this assumption. In particular, "random" *reductions* are poorly supported, as conventional caches lack native mechanisms to merge updates efficiently or to retain relevant data in limited storage.

In the next sections, we introduce cache architectures supporting *reduction* operations, specifically designed to handle irregular data. These architectures generalize the concept of a data-cache to include *reduction*-aware mechanisms, ensuring correctness while improving bandwidth efficiency.

In the following sections, we introduce cache architectures with native *reduction* support, specifically designed to handle irregular data. These architectures extend the classical cache model with *reduction*-aware mechanisms, improving bandwidth efficiency under irregular access patterns.

CONCLUSION

Generic data-caches efficiently exploit regular access patterns but **fail to address the irregularity** and accumulation requirements of sparse data computing. These limitations motivate the introduction of ***reduction*-aware cache architectures**, capable of handling irregular data.

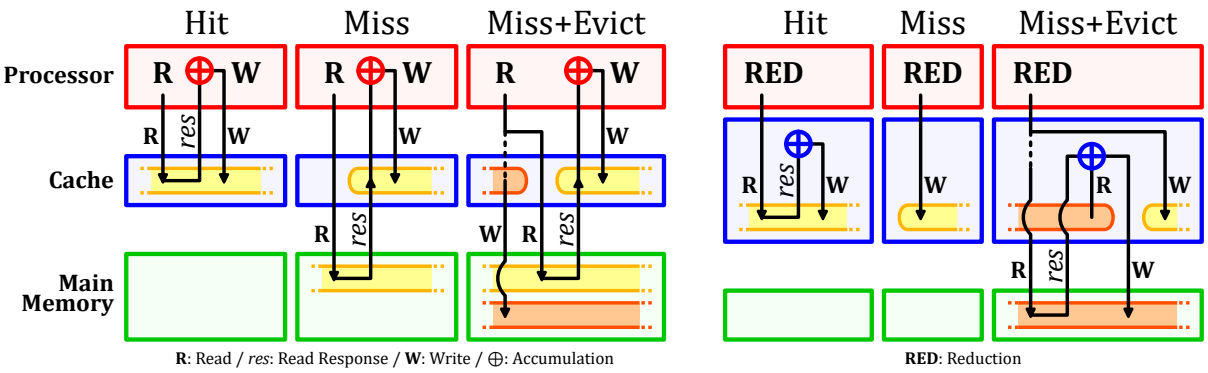
3.3 Reduction Cache

PERFORMING a *reduction* in hardware requires a read from memory, followed by an accumulation and a write back. This operation is denoted $RED(key, value)$ [45] with the *key*

being the position in the output vector c. In Figure 3.1a, we show three cases that occur in a generic data-cache when a *reduction* is performed. If the vector entry (c_{key}) hits in the cache, the *update* is performed by the processor (in *red*) with no external memory access (on the left in the figure). When there is a miss, the operation is blocked until the read from main memory completes (in the middle of the figure). Potentially, the miss will also trigger an eviction, which exacerbates the blocking condition, because a cache-line must first be written back, before the read from main memory can be performed (on the right in the figure). In scientific applications, the matrices are extremely large and only a small portion of the vector c can reside in the cache, thus the *miss+evict* case arises frequently.

A *reduction cache*, such as [45] or [39], is a data-cache which accepts *reduction* instructions (*RED*) from the processor and performs accumulations locally (near-memory). In the case of a hit, see Figure 3.1b, the *reduction* takes place in the cache (in *blue*, on the left in the figure). For a miss, the cache allocates a new line with zeros, and simply inserts the value (in the middle of the figure). In this case, the cache-line is inconsistent with main memory. Only when the line needs to be evicted, is a read performed (on the right in the figure). In the cache, the values in the line are accumulated with the main memory data, and the result is written back.

As seen in the figure, a *reduction cache* reduces the number of main memory accesses in the case of a miss. Instead of having the classic ascendant policy where data is stored in the cache to be pulled-up again by the processor latter-on, the *reduction cache* has a descendant policy: data is written/updated in the cache to be pushed-down to memory latter-on. Instead of two memory transactions that can stall the processor, the *reduction cache* only needs a single "fire-and-forget" instruction, meaning it can be issued without waiting for an answer, thus avoiding stall issues. At the end of the algorithm, the *reduction cache* must be flushed.



(a) Generic Cache (Write-Back) **(b)** Reduction Cache

Figure 3.1: Read (R) and Write (W) Transactions of a *Reduction* Operation in Generic and *Reduction* Caches

DEFINITION

A ***reduction cache*** is a data-cache containing an arithmetic unit and a dedicated support for *reduction* operations, via single "fire-and-forget" *RED* instructions.

CONCLUSION

In order to process efficient *reduction* operations, a ***reduction cache*** can be used to **avoid unnecessary memory transactions and stalls**, by offloading the accumulation near-memory, and updating the main memory only at eviction.

3.4 Outer-Product SpMV with a Reduction Cache

A widely used sparse format to compress the non-zero elements in a matrix A is CSC [44], where A is a sparse matrix of dimensions (M, N) with a number of non-zero elements nnz. As seen in Chapter 1, it consists of three tables (val, idx, ptr) used to store non-zero values, row indices and column positions. When performing an `OP` SpMV ($A \times b = c$) with this format (Algorithm 3.1), we see that a *RED* is needed to accumulate the output vector c (line 6).

Moreover, A is read sequentially while c is updated "randomly". In fact, the *key* is determined via an indirection with an access to idx.

Algorithm 3.1: *Outer-Product* SpMV – Single Threaded

```
1 for n ∈ [[0, N − 1]] do                          // Loop over the columns of A
2     for pos ∈ [[ptr_n, ptr_{n+1} − 1]] do
3         key ← idx_pos;                                          // Row index
4         val ← val_pos × b_n;                    // val = A_{key,n} × b_n
5         RED(key, val);                                          // c_key += val
```

In memory, A is stored as three dense tables, but conceptually, as we perform the **OP** multiplication, the matrix is traversed from top to bottom, then left to right. A row in the matrix corresponds to a *key*, or equivalently, a position in the output vector c. In Figure 3.2, we show an example of a sparse matrix with a few non-zero values (① - ⑥). As we traverse the matrix, we first perform a *reduction* for the value on row key_{10}, then on row key_{20} and then key_{30}. Each of these results in a *RED* on c. As we continue, we then hit another value ⑤ on the row key_{10} which must be *reduced* with an existing entry ① (as shown on the left of Figure 3.2, *Conceptual View*).

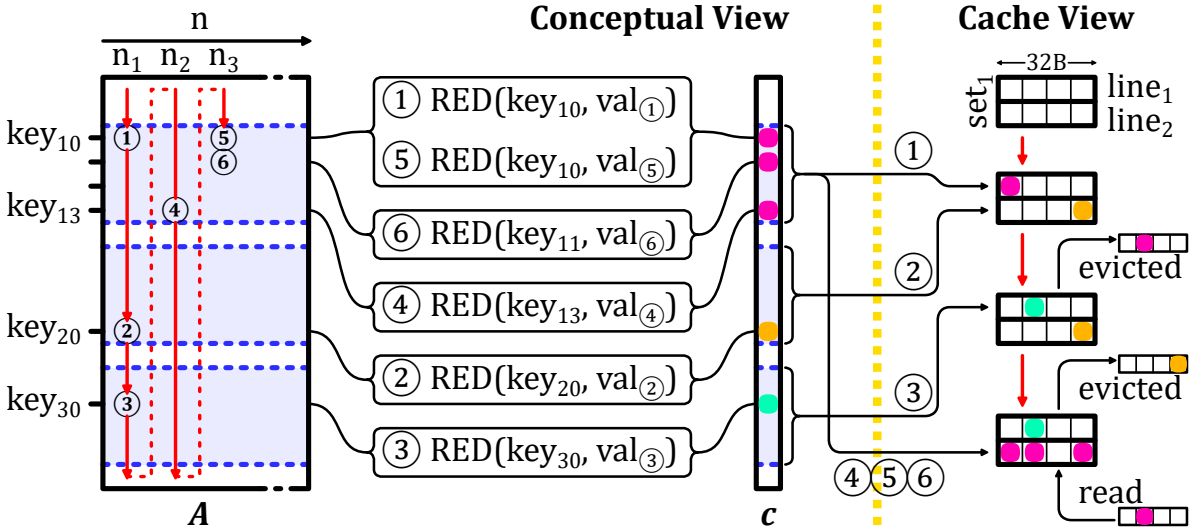


Figure 3.2: Traversal of Non-Zero Elements of Input Matrix for **OP** SpMV (left) and Impact on Cache (right)

Let us consider how c is stored in a two-*way* *set*-associative cache with 32B lines. We assume that all the *keys* map to set_1. When ① is updated, there is a miss and $line_1$ is allocated for key_{10}. Similarly with ②, $line_2$ is allocated for key_{20}. When key_{30} comes along, an eviction must be performed. We assume $line_1$, holding key_{10}, is evicted. We then come to ④ which has key_{13}, adjacent in memory to key_{10}, and thus stored in the same line. This line must be brought back to process ④, ⑤ and ⑥.

This example illustrates how a conventional cache can be inefficient. First, we note the line holding key_{10} was evicted, although it was frequently accessed after ①. This eviction was triggered by key_{30}, which was only accessed once. Furthermore, for key_{20} and key_{30}, a full line was loaded, only to modify a single entry. A "smarter" cache would have maintained the line with key_{10} to avoid an unnecessary main memory access. Furthermore, this "smarter" cache would not have allocated full lines for key_{20} and key_{30} to only modify a single word.

The globally low local and temporal localities avoid caches, and even *reduction caches*, to take advantage of any reuse opportunity. However, sparse matrices have various types of structure which can have reuse potential on some parts of the matrix. In the example in Figure 3.2, the first rows (on key_{10} to key_{13}) are denser than the rest of the matrix. A "smarter" cache would prefer to focus on these rows, keeping the corresponding cache-line and accumulating every elements before a final eviction. On the contrary, the sparser rows of the matrix would not stayed long in the "smarter" cache, having low priority to avoid thrashing data from denser regions.

We focus on the case of banded matrices which have a denser region around the diagonal. For a *reduction cache* to work efficiently with such matrices, the dense band should be prioritized, and the isolated elements out of the band should be processed separately to avoid disturbance. This is the main idea of SpDCache (Paper II), which is a *reduction cache* with dedicated memory regions for the in-band and out-of-band of a matrix. We describe its behavior int the following sections.

CONCLUSION

When computing an `OP` **SpMV**, a **conventional *reduction cache* is not enough** for the available cache memory to be efficiently utilized. Compared to the size of a cache, matrices have too large dimensions for *keys* to be reused from one column iteration to an other, leading to mostly empty cache-lines being prematurely evicted.

We propose **SpDCache**, a *reduction cache* with dedicated memory regions, **leveraging matrix structure** with denser and sparser regions, such as **banded matrices**.

3.5 Architecture of SpDCache

SPDCACHE is developed to replace a L1 generic data-cache. Thus, it has to function as a generic cache when receiving classical memory instructions from the processor, such as reads and writes. Indeed, when processing an `OP` SpMV, the sequential reads to the matrix A and the vector b should not bypass the cache hierarchy which takes advantage of spatial locality. Only, the output vector c suffers from "random" *reductions* that should be processed with a region-wise *reduction cache*. SpDCache can be extended on a multilevel memory hierarchy, as well as a multicore architecture (see Chapter **??**). However, our study (up to Chapter **??**) focus on a single level cache hierarchy with a single core, to focus on the SpDCache behavior.

As shown in Figure 3.3, SpDCache is divided into three regions. The first, *Generic Region Cache* (*GRC*), is for all memory transactions unrelated to the output vector c, including reads of A and b. It is a classic, *set*-associative cache and not our focus, but is required for the processor to operate normally. The other two regions, called *Dense Region Cache* (*DRC*) and *Sparse Region Table* (*SRT*), are able to perform *reductions* on c. The *DRC* works as a line-wise, *set*-associative *reduction cache* to hold high density data in the *In-Band Region* of the matrix (*IBR*, *orange*). The difference with the *GRC* is that it is able to perform *reductions* and it is dedicated to the *banded* region, so that spurious accesses do not provoke undesired evictions. Finally, the *SRT* works as an element-wise, fully-associative table whose role is to handle sparse regions, such as the *Out-of-Band Region* in the matrix (*OBR*, *blue*).

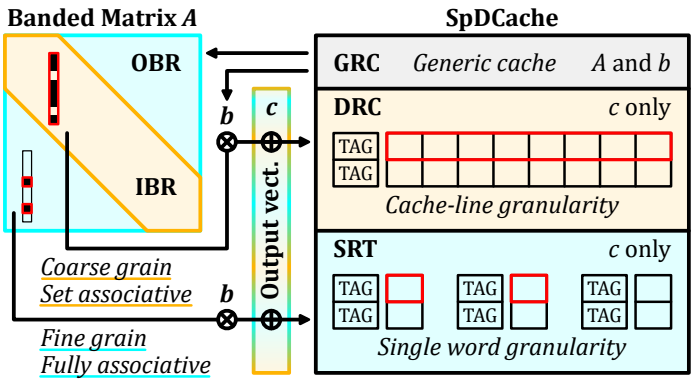


Figure 3.3: Data-Cache Partitioning into Regions (*GRC*, *DRC*, *SRT*)

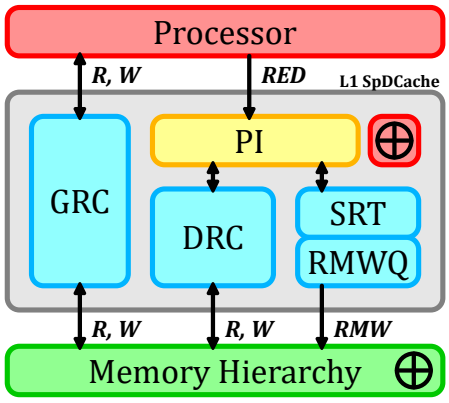


Figure 3.4: Simplified View of the Memory Hierarchy with SpDCache

Figure 3.4 shows a simplified view of a memory hierarchy with SpDCache. The *Processor Interface* (*PI*) takes *reduction* instructions, *RED(key, value, region)*, from the processor and dispatches them to the *DRC* and *SRT*, depending on the *region* argument. The *DRC* communicates with the memory hierarchy using read and write transactions over entire cache-lines, to propagate the local *updates* to the downstream memory. As *SRT updates* are for isolated elements, it does not make sense to bring a full line into the L1 to modify a single word.

The *SRT* thus propagates *updates* by issuing atomic *Read-Modify-Write* transactions (*RMWs*), which must be processed remotely and close to the main memory. Existing protocols such as AXI-4 [1] already support atomic transactions for certain operations (logical, integer add, min, max) and these would need to be extended to include floating point addition. The *RMWs* from the *SRT* are temporarily buffered in a small queue (*RMWQ*), to avoid congestion.

SpDCache is driven by three key design principles. First, we ensure a given *key* is in only one region, to avoid coherency problems. Second, the *DRC* is given priority over the *SRT*, to enable coalescing. Third, we avoid bringing a full line from memory to the L1 to simply update a few entries.

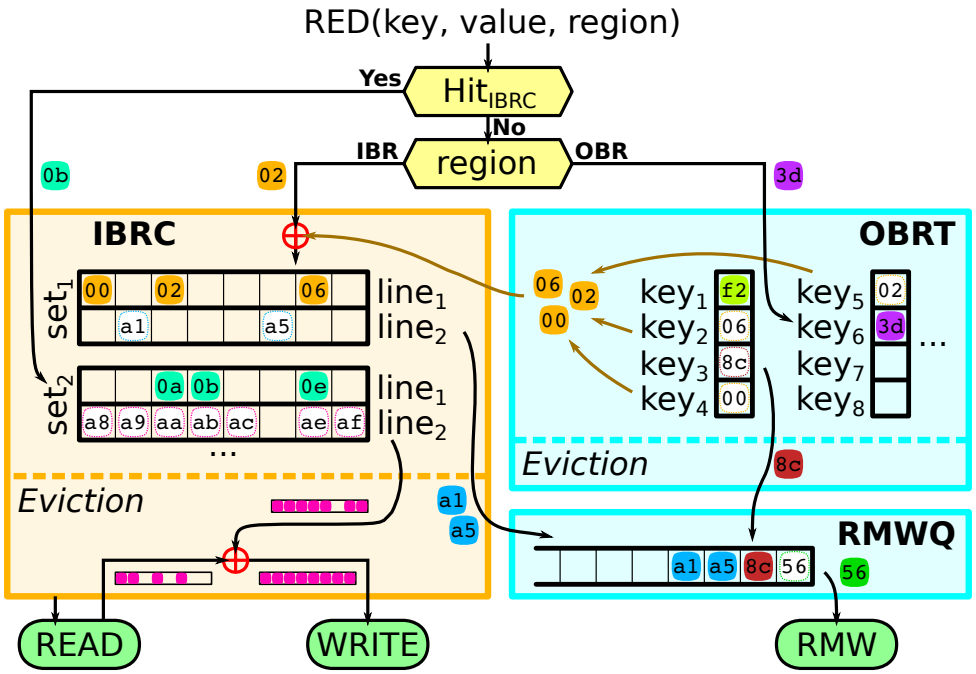


Figure 3.5: Internal Operation of SpDCache with Examples **TODO: update region names**

In Figure 3.5, we describe the operation of SpDCache, via a series of scenarios, each shown in a different color. When a new *reduction RED(key, value, region)* is received from the processor, we test for a hit in the *DRC*, regardless of the *region* argument. If it hits, the value (case `0b` – *turquoise*) can be accumulated in the corresponding cache-line of the *DRC*. We give this priority to the *DRC* as part of the mechanism to avoid duplication.

If there is a miss in the *DRC*, then the target region is determined based on the *region* argument of the *RED* transaction. If it is *IBR*, then it is necessarily a miss case, and the value (`02` – *orange*) must be inserted into a free cache-line of the *DRC*. When we allocate this line, we ensure that it does not create a duplication of existing entries in the *SRT*. If there is duplication, as in our example, these entries (`00`, `02`, `06`) are deleted from the *SRT* and merged into the newly allocated cache-line ($set_1, line_1$).

If the target region is the *OBR*, then the *reduction* (`3d` – *purple*) is processed in the *SRT*, either allocating a new entry (miss case) or accumulating with an existing entry (hit case).

Now, we describe the eviction scenarios. In the *DRC*, the line to evict is selected ($set_2, line_2$ – *pink*), a read to main memory is performed, the values from the evicted cache-line are accumulated and a write to main memory is issued. If the evicted cache-line only has a few non-zero elements and the *RMWQ* has enough free entries, it is a special case. To avoid bringing a full line from main memory, the non-zero values (`a1` and `a5` – *blue*) are converted to atomic *RMWs* and pushed onto the *RMWQ*. The decision to convert evicted cache-lines to atomic *RMWs* is based on a configurable density threshold. Previous work [39] used a threshold of one non-zero word per eight words in a cache-line. Given our queuing mechanism to handle congestion, we use a more aggressive threshold of three non-zero words per cache-line.

The width of the *band* (w_B), must be selected so that one column of the *band* remains blocked in the *DRC*, otherwise, this region of the cache is always thrashing. This is our approach to selecting w_B, which we detail in the next chapter. Note that this approach selects w_B based on the *DRC* size, independently of the matrix and its structure.

In the *SRT*, an eviction simply consists of moving the element from the table to the *RMWQ*

(8c – *brown*). If the *RMWQ* is not full, *SRT* evictions are issued in advance when the *SRT* table is nearly full, using an other configurable threshold sets to half the size of the queue in our experiments. The *RMWQ* issues atomic *RMWs* when it is not empty (56 – *dark green*).

NOTE

SpDCache is not limited to banded matrices. Any sparse matrix with a denser region can benefit from this architecture, by adapting the definition of the *band* accordingly, and making sure that the blocking effect in the *DRC* is preserved.

CONCLUSION

SpDCache is a data-cache splits in **three regions**:

- *GRC*, a **generic** cache region processing reads and writes of A and b,
- *DRC*, a *reduction cache* region dedicated to *reductions* on c when computing the **band** (*IBR*),
- *SRT*, a *reduction cache* region dedicated to *reductions* on c when computing the isolated **out-of-band** elements (*OBR*).

The two last regions use **adapted memory transactions** to avoid transferring unnecessary data.

3.6 High Level Evaluation of SpDCache

We modelled the behavior of SpDCache, based on the flowchart in Figure 3.5, using a transaction level model in Python, for a single-core, single-thread system executing a `OP` SpMV kernel (Algorithm 3.1). We do not model timing, as the order of the memory accesses determines the behavior of the cache (hits, misses, evictions), which depends only on the SpMV algorithm. The model checks the final numerical result and reports the main memory traffic associated with the *updates* to the output vector (*Output Memory Traffic*), where we count the bytes transferred (64B for each read and write, and 8B per atomic *RMW*, with our configuration). For each write transaction, it tracks how many words in the line were actually used (*Evicted Cache Line Utilization*) and it tracks how many words in the cache held useful data (*Cache Memory Utilization*). We consider a word in the cache to be useful if it is updated or inserted by the processor. Our goal is to validate the principle of SpDCache using real application matrices from the SuiteSparse Matrix Collection [33].

As baselines, we implemented a fully generic cache (GC) that handle reads and writes for both inputs (A, b) and output (c), following the data movements shown in Figure 3.1a, and a simple *reduction cache* (RedC) that processes *RED* transactions in a dedicated region using the flow shown in Figure 3.1b. In the RedC, 25% of the memory is allocated for the inputs and the other 75% for *REDs* on c. In SpDCache (SpDC), 25% is allocated for the *GRC*, 50% for the *DRC* and 25% for the *SRT*. In the next section, we present more results with varying region sizes, showing that this configuration is a good compromise. All three caches use a *Least Recently Used* (*LRU*) eviction policy, except the *SRT* region of SpDC which uses a *random* policy. All three caches have a total data storage of 32 KiB, and we do not account for the *TAG* storage. The *set*-associative regions have 4 *ways* and 64B cache-lines (8 *double* words per line).

We simulated 48 matrices that have been used by the state-of-the-art accelerators [4, 24, 40, 45, 46, 56, 57], which are listed in Appendix D. We excluded matrices whose output vector c fits entirely in the cache (fewer than 4096 elements), since they do not stress the cache. In Table 3.1, we report the results for all three caches, along with statistical indicators. Most, but not all, of the matrices have a dense band and even some matrices without an obvious band benefit from the blocking effect in the *DRC*.

As expected, the simple *reduction cache* (RedC) decreases the main memory traffic to 89.2% of the GC baseline. SpDCache has significantly better performance than RedC, reducing the traffic to 24.5%. SpDCache thus achieves a 4.1× decrease in memory traffic compared to GC, and a 3.6× decrease compared to RedC, on average over the set of matrices. SpDC outperforms GC on all matrices, and RedC on 93.8% of them.

The benefits of SpDCache can be partly explained by the improved utilization of the cache-lines. On average, a cache-line evicted from SpDC has 7.21 modified words, compared to 3.56 and 3.60 with the baselines. This improved utilization is the result of blocking the dense region in the *DRC*, where multiple *reductions* have the time to accumulate in the cache before being evicted. The *SRT* also contributes to this effect, as it captures irregular accesses outside

Table 3.1: Decrease in Memory Traffic and Improvement in Cache Utilization from Python Simulation

	Output Memory Traffic			Evicted Cache Line Utilization			Cache Memory Utilization (%)		
	RedC	SpDC		GC	RedC	SpDC	GC	RedC	SpDC
	%/GC	%/GC	% RMW	Avg	Avg	Avg	Avg	Avg	Avg
Geometric Mean	89.2	24.5[a]	40.5	3.60	3.56	7.21[b]	36.4	45.8	75.2[c]
Standard Deviation	29.8	20.9	33.8	2.45	2.77	2.08	10.9	32.3	18.6
Minimum	28.5	7.00	0.00	1.06	1.00	4.36	16.6	11.9	30.0
Maximum	172	98.4	100	8.00	8.00	8.00	49.6	97.5	98.9
First Quartile	76.0	11.4	35.9	1.76	1.73	6.03	28.4	22.0	65.5
Third Quartile	115	46.6	91.7	6.60	7.65	8.00	48.6	88.8	93.9
Outperform GC	45.8	100	Ø	Ø	45.8	91.7	Ø	64.6	100
Outperform RedC	Ø	93.8	Ø	Ø	Ø	85.4	Ø	Ø	77.1

[a] *Decrease by 4.1× compared to GC, and 3.6× compared to RedC.*
[b] *Increase by 2.0× compared to GC, and 2.0× compared to RedC.*
[c] *Increase by 2.1× compared to GC, and 1.6× compared to RedC.*

the band, which would otherwise evict useful data from the *DRC*. The overall cache memory utilization is also significantly improved, with 75.2% of the cache holding useful data in SpDC, compared to 45.8% and 36.4% for RedC and GC, respectively.

We need to consider an additional metric, the percentage of traffic sent as atomic *RMW* transactions (% RMW). If this value is too high, it means we have simply shifted the workload to the lower level caches, which inherently have limited compute capability. In mean, 40.5% of the memory traffic of SpDCache is composed of atomic *RMWs*, which is acceptable given the significant overall reduction in memory traffic. However, this ratio varies widely between matrices. There is some matrices where most of the traffic is composed of atomic *RMWs*, meaning that most of their non-zero values are outside the band. For such matrices, which are basically not banded, SpDC still greatly decrease the memory traffic, but better performance could be achieved by choosing a matrix-fitting region partitioning strategy. For example, the matrix *cit-Patents* has no elements in the band, and all non-zero values are scattered under the bottom part of the matrix (an unusual extreme case). While having a memory traffic decreased to 7.2% compared to GC, it could benefit from a better use of the *DRC* by choosing a horizontal band covering the non-zero elements, with correct sizing to leverage the blocking effect. On the other hand, some matrices have very low *RMW* usage, indicating that most non-zero values are within the band and efficiently handled by the *DRC*. In this case, 25% of the available memory was unused. The *SRT* size could be adjusted to gain performance.

CONCLUSION

SpDCache achieves a 4.1× **decrease in memory traffic** on the output vector c compared to a generic cache, over a set of 48 matrices. It also **outperforms a simple *reduction cache*** by 3.6×, showing the importance of having separate dedicated cache regions for the in- and out-of-band of a matrix.

3.7 State-of-the-Art Comparison and Region Sizing

To compare SpDCache with the state-of-the-art, we simulated PHI [39], the existing solution that is closest to our proposal, using the same methodology as in the previous section. PHI is a *reduction*-aware cache that uses a single region to process *RED* transactions (see Section **??** from previous chapter). PHI works as if our three regions, *GRC*, *DRC* and *SRT*, where merged together, using different technics to handle *reductions*, load and store operations in the same cache address space. It also handles isolated elements, but do not consider the matrix structure. Conversely to SpDC, PHI relies on the use of the *deferred update* version of the `OP` SpMV algorithm (see Algorithm 1.8). We thus adapted our simulation to use this algorithm for PHI. To ensure a fair comparison, we allocated the same amount of memory to PHI as for SpDC and the baselines (32 KiB). In Table 3.2, we report the results for PHI alongside our

previous results (columns 2, 3, 6 and 7).

Table 3.2: All Results from Python Simulation

GMeans (48 mat.)	**GC**	**RedC**			**PHI**	**SpDC**					
sets GRC	*128*	*32*	*8*	*4*	*128*	*32*	*8*	*8*	*4*	*4*	*4*
sets DRC	*Ø*	*96*	*120*	*124*		*64*	*104*	*112*	*92*	*116*	*122*
sets SRT	*Ø*	*Ø*	*Ø*	*Ø*		*32*	*16*	*8*	*32*	*8*	*2*
Output Memory Traffic (% over GC)	100 ■1.0	89.2 ▼1.1	81.6 ▼1.2	80.3 ▼1.3	50.4 ▼2.0	24.5 ▼4.1	24.3 ▼4.1	25.5 ▼3.9	23.2 ▼4.3	25.3 ▼4.0	32.0 ▼3.1
Ratio of RMWs in Memory Traffic (%)	Ø	Ø	Ø	Ø	Ø	40.5	35.7	36.5	36.5	36.2	41.1
Total Memory Traffic (% over GC)	100 ■1.0	93.9 ▼1.1	91.9 ▼1.1	93.2 ▼1.1	98.8 ■1.0	56.1 ▼1.8	56.6 ▼1.8	56.9 ▼1.8	57.7 ▼1.7	58.2 ▼1.7	59.2 ▼1.7
Evicted Cache Line Utilization (max: 8)	3.60 ■1.0	3.56 ■1.0	3.66 ■1.0	3.67 ■1.0	5.00 ▲1.4	7.21 ▲2.0	7.21 ▲2.0	7.23 ▲2.0	7.22 ▲2.0	7.22 ▲2.0	7.25 ▲2.0
Cache Memory Utilization (%)	36.4 ■1.0	45.8 ▲1.3	47.0 ▲1.3	47.1 ▲1.3	30.6 ▼0.8	75.2 ▲2.1	68.3 ▲1.9	65.0 ▲1.8	72.5 ▲2.0	65.1 ▲1.8	60.9 ▲1.7

As expected, PHI achieves a significant reduction in memory traffic compared to the GC, down to 50.4% on average. This represents a 2.0× decrease compared to the GC, which is consistent with the results reported in the original PHI paper [39], and a 1.8× decrease compared to the RedC. However, SpDCache still outperforms PHI, cutting memory traffic by 2.1×. The improved performance of SpDC can again be explained by the better utilization of the cache-lines, with 7.21 useful words per evicted line, compared to 5.00 for PHI. The overall cache memory utilization is also significantly better with SpDC, with 75.2% of the cache holding useful data, compared to only 30.6% for PHI.

We also explored different region sizing for SpDCache, varying the sizes of the *GRC*, *DRC* and *SRT* while keeping the total cache size constant at 32 KiB. The results are also reported in Table 3.2. Reducing the size of the *GRC* from 8 KiB (32 *sets*) to 2 KiB (8 *sets*) and 1 KiB (4 *sets*) has little impact on performance, as the *GRC* is mainly used for temporary storage of input vector elements. However, reducing it under 1 KiB would increase the total memory traffic, as more frequent evictions would occur before the data is used, an effect already noticeable when going from 2 KiB to 1 KiB. With constant *GRC* size, when reducing the size of the *SRT*, which means allocating more space to the *DRC*, we observe a slight increase in memory traffic, as the *SRT* becomes less effective at capturing irregular accesses. Thus, the *SRT* sizing appears to be more critical than the *DRC* sizing. We will explore the reasons behind this effect in the next chapter. Overall, fine tuning the region sizes can lead to small performance improvements, but the original sizing of 8 KiB for the *GRC*, 16 KiB for the *DRC* and 8 KiB for the *SRT* appears to be a good compromise for the set of matrices we evaluated.

About the RedC, we also explored different sizing strategies. We observed that allocating more than 75% of the cache (96 *sets*) to store *RED* transactions slightly improves performance, as it increases the likelihood of capturing multiple updates to the same output element before eviction, without overly compromising the storage of input elements.

CONCLUSION

SpDCache outperforms PHI, a state-of-the-art *reduction* cache, by achieving a 2.1× decrease in memory traffic, thanks to its dedicated regions for in- and out-of-band accesses that improve cache-line and overall cache utilization. Additionally, we found that the simple 25/50/25% region sizing of SpDCache provides good performance with only minor variations observed when adjusting the sizes of the *GRC*, *DRC*, and *SRT*.

3.8 Conclusion

We have presented the concept of SpDCache, a *reduction cache* that has optimized storage and compute techniques for the sparser and denser regions of banded matrices. This approach does not reduce the number of compute operations, however, it significantly decreases memory traffic (4.1×). These benefits can be attributed to three techniques: (1) blocking the dense part of the band in the *DRC*, (2) using fine-grained storage in the *SRT* and (3) off-loading the *reductions* for isolated elements to the downstream caches, to prevent local evictions and reduce traffic.

Exploiting the coarse-grain structure of matrices is key to design hardware optimized for sparse matrix kernels. In the next chapter, we demonstrate this with a more theoretical approach, allowing us to go further into SpDCache analysis, and justify appropriate region sizing, before going into a cycle-accurate evaluation.

TAKEAWAYS

Reduction cache:

- Support a *reduction* instruction: $RED(key, value)$
- Accumulate near memory
- Reduce the number of memory transactions

SpDCache:

- Take advantage of coarse grain structure of matrices: matrix regions such as *IBR* and *OBR* for banded matrices
- Three cache memory regions:
  - *GRC*, generic region cache for generic instructions
  - *DRC*, dense region cache for *reductions* in the *IBR*
  - *SRT*, sparse region table for *reductions* in the *OBR*
- Adapted memory transactions: cache-line wise versus element wise for the *DRC* and the *SRT*, respectively
- Overall: greatly reduce memory traffic

Conclusion

Contents

1 TODO

TODO

Appendices

Contents

A Sparse Formats

TODO

COO The simplest *sparse format* is called *COO* (*COOrdinate*) [13, 44, 48]. It consists in three separate tables, one filled only with the non-zero elements of the matrix, a row index table containing the row of the corresponding elements, and similarly a column index table (see Figure **??**). The two latter are *metadata* tables and all three tables are of size *nnz*. In this format, the elements are not necessarily sorted, the memory footprint does not depend on the structure of the matrix and all elements are stored independently of each other.

CSR & CSC The most commonly used *sparse format* is called *CSR* (*Compressed Sparse Row*) [6, 13, 44, 48]. It is similar to the *COO* format except that the row index table is replaced by a set of indices that point to the position of the start of each row in the column index table (see Figure **??**). The size of this table is the number of rows plus one, and, in practical cases, the memory footprint is smaller than with *COO*. However, the non-zero elements need to be sorted in *row-wise* order. This format has a *column-wise* variant called *CSC* (*Compressed Sparse Column*) where the column index table stores the positions of columns instead of rows. These two formats are symmetrical but with *CSR*, it is the traversal of rows that is efficient because they are sequential whereas in *CSC*, column traversal is efficient.

BCSR Even in sparse matrices, there can be regions, or blocks, that have a large fraction of non-zero values. In this case, the *BCSR* format [6], which is an extension of the *CSR* format with *tiling*, can be attractive. We use the term *tiling* to describe a level of hierarchy in a storage format. That is, a *tile* refers to a sub-matrix, which itself may be dense, as in the case of *BCSR*, or which could be sparse. With the *BCSR* format, we simply replace the non-zero elements of the *CSR* format with dense, fixed-size sub-matrices (see Figure **??**). Within these sub-matrices, it is possible to have some zero elements. The advantage of the *BCSR* format is that the dense sub-matrices can be processed using techniques optimized for dense matrices, particularly using vector units or GPUs [25]. Compared to the *CSR* format, *BCSR* makes it possible to accelerate the computation within the dense *tiles*, but this comes with the overhead of storing some zero values. Not all matrices are well suited to the *BCSR* format.

DIA For diagonally structured matrices ($\forall (i,j) \in [\![0,M]\!] \times [\![0,N]\!],\ a_{i,j} = 0\ for\ |i-j| > K$), a format called *DIA* (*DIAgonal*) is particularly well suited [6, 13, 44]. It consists in storing the values along the $2 \times K + 1$ diagonals in a table and keeping information about the position of the diagonals in the matrix (see Figure **??**). The organization of the non-zero data and selection of the diagonals can be tailored to the matrices and access patterns. This format is well suited to diagonal matrices, has a low overhead for storing indices and can be combined with other formats to better fit more complex structures.

ELL Another format called *ELL* (*ELLPACK*) [6, 13, 30, 44] is well suited when the number of non-zero elements per row can be bounded by K. In this case, the non-zero values in a $M \times N$ matrix can be stored as a dense $M \times K$ matrix, augmented with an index table, of the same dimensions, which gives the column position of each non-zero value (see Figure **??**). This format is not efficient if the number of non-zero elements per row is highly variable, as many entries in the $M \times K$ table are unused. This format is well suited to GPUs [11, 50] since it transforms a sparse matrix into a dense one in memory.

HYB The *HYB* format (HYBrid) [7] is an example of a format which combines *ELL* and *COO*. The *ELL* format can be used, but with a value of K (the width of the value table) that is smaller than the maximum non-zero entries per row in the original matrix (see Figure **??**). For the small number of rows where the table lacks a sufficient number of columns, the *COO* format can be used to store the extra values. In this way, the storage efficiency of *ELL* is improved, and it can be applied to a broader class of matrices.

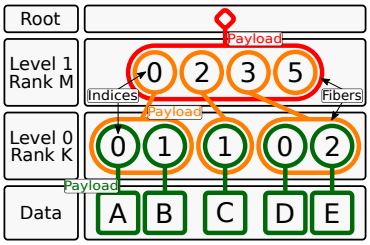


Figure 3.6: Example of the *FiberTree* Representation with CSR Format

B A Generic Representation for *Sparse Formats*

A visual representation of compressed matrices, called *FiberTree* [47], is helpful to understand these sparse formats. Each *rank* of a matrix represents a level of a tree where the nodes are filled with either the metadata of the child nodes or the matrix data which can only appear in the leaves (see Figure 3.6). With this representation, we clearly identify the number of *ranks*, which define the *fibers*. A *fiber* is the generic term to describe a one-dimensional stream of data or metadata, which corresponds to a row or column in the case of a two-dimensional matrix. A stream of indices is a *fiber* of metadata.Figure 3.6 presents, as an example, a *FiberTree* representation of the matrix of Figure **??**, matching the CSR format: each *rank* in the tree corresponds to a *fiber* stored in the table *Row PTR* or *Col IDX* of Figure 1.3b.

In a recent work [53], a systematic nomenclature for *sparse matrix (and tensor) formats* was proposed, based on *FiberTree* representation. In fact, in the example of Figure 3.6, the *fibers* are compressed with different methods. Thus each *rank* can be characterized by a label indicating how it is compressed: *Uncompressed* (*U*), *Coordinate Payload* (*CP*), *Uncompressed Offset Pairs* (*UOP*). *U* defines *ranks* where all data values are stored contiguously in memory. *CP* defines *ranks* that store the indices of each payload, a payload being either a non-zero element or a sub-matrix[1]. *UOP* defines *ranks* that store pointers (offsets) to delimit sub-matrices of the next *rank*[2]. These are the three main ones as they are used in the most common sparse formats but many others exist or can be created.

With this nomenclature, the sparse formats can be classified according to their *rank* compression methods. For example, the CSR format is denoted as *(UOP, CP) row-major* since its *Row PTR* table stores pointers (offsets) to rows. The CSR and CSC formats are both of type *(UOP, CP)*, one being *row-major*, the other *column-major*. The COO format is denoted as CP^2 because there are two coordinates (row and column) for each non-zero element, as it is a *CP fiber* having two indices per data element instead of one. User defined formats can also be easily described and named. For example, a user could define a storage format with a nested CSR. That is, each non-zero element in the outer CSR, corresponds to a sub-matrix, itself stored in CSR format. Such a representation would be labeled *(UOP, CP, UOP, CP)*. A similar nomenclature is proposed by TACO [13]. It is more adapted to code generation applications, where the *FiberTree*-based nomenclature better describes the memory storage of the sparse format. For example, *U* corresponds to *Dense*, *CP* to *Singleton*, the pair *(UOP, CP)* is translated as *Compressed* while *UOP* alone corresponds to *Offset*. We chose to adopt the *FiberTree*-based nomenclature in this article since it is more suited to a hardware context.

[1] In this section, we use the term sub-matrix but this is extended to the notion of *tiling* in Section 1.4.1.
[2] If some sub-matrices are empty, pointers will be repeated, hence 'uncompressed'.

C Extended Overview of the State-of-the-Art Accelerators for Sparse Computing

TODO

D Sparse Matrices Used for the Evaluations

TODO: table of all 62+ matrices, with characteristics, which eval they were used for

E Analytical Model of a *Reduction Cache* Computing an `OP` SpMV Kernel

TODO

F Python Model of a Data-Cache with Support for *Reductions*

TODO

G Statistics

TODO (get infos before end of contract)
(commits, nb. of lines, bugs, timeline -> into figures?)

List of Figures

List of Tables

List of Algorithms

List of Source Code

Bibliography

[1] ARM IHI 0022. 2023. ARM – AMBA AXI Protocol Specification. (Sept. 2023). https://developer.arm.com/documentation/ihi0022/latest https://developer.arm.com/documentation/ihi0022/latest.

[2] Venkitesh Ayyar, Evan Weinberg, Richard C. Brower, M.A. Clark, and Mathias Wagner. 2023. Optimizing Staggered Multigrid for Exascale performance. In *Proceedings of The 39th International Symposium on Lattice Field Theory — PoS(LATTICE2022)*, Vol. 430. 335. https://doi.org/10.22323/1.430.0335

[3] Ariful Azad, Aydin Buluç, and John Gilbert. 2015. Parallel Triangle Counting and Enumeration Using Matrix Algebra. In *2015 IEEE International Parallel and Distributed Processing Symposium Workshop*. 804–811. https://doi.org/10.1109/IPDPSW.2015.75

[4] Daehyeon Baek, Soojin Hwang, Taekyung Heo, Daehoon Kim, and Jaehyuk Huh. 2021. InnerSP: A Memory Efficient Sparse Matrix Multiplication Accelerator with Locality-Aware Inner Product Processing. In *2021 30th International Conference on Parallel Architectures and Compilation Techniques (PACT)*. 116–128. https://doi.org/10.1109/PACT52795.2021.00016

[5] Ricardo Baeza-Yates and Alejandro Salinger. 2010. *Fast Intersection Algorithms for Sorted Sequences*. Springer Berlin Heidelberg, Berlin, Heidelberg, 45–61. https://doi.org/10.1007/978-3-642-12476-1_3

[6] Richard Barrett, Michael Berry, Tony F. Chan, James Demmel, June Donato, Jack Dongarra, Victor Eijkhout, Roldan Pozo, Charles Romine, and Henk van der Vorst. 1994. *Templates for the Solution of Linear Systems: Building Blocks for Iterative Methods*. Society for Industrial and Applied Mathematics. https://doi.org/10.1137/1.9781611971538

[7] Nathan Bell and Michael Garland. 2009. Implementing Sparse Matrix-Vector Multiplication on Throughput-Oriented Processors. In *Proceedings of the Conference on High Performance Computing Networking, Storage and Analysis* (Portland, Oregon) *(SC '09)*. Association for Computing Machinery, New York, NY, USA, Article 18, 11 pages. https://doi.org/10.1145/1654059.1654078

[8] Achi Brandt and Dorit Ron. 2003. *Multigrid Solvers and Multilevel Optimization Strategies*. Springer US, Boston, MA, 1–69. https://doi.org/10.1007/978-1-4757-3748-6_1

[9] Sergey Brin and Lawrence Page. 1998. The anatomy of a large-scale hypertextual Web search engine. *Computer Networks and ISDN Systems* 30, 1 (1998), 107–117. https://doi.org/10.1016/S0169-7552(98)00110-X Proceedings of the Seventh International World Wide Web Conference.

[10] Aydın Buluç, John Gilbert, and Viral B. Shah. 2011. *13. Implementing Sparse Matrices for Graph Algorithms*. Society for Industrial and Applied Mathematics, Chapter 13, 287–313. https://doi.org/10.1137/1.9780898719918.ch13

[11] Wei Cao, Lu Yao, Zongzhe Li, Yongxian Wang, and Zhenghua Wang. 2010. Implementing Sparse Matrix-Vector multiplication using CUDA based on a hybrid sparse matrix format. In *2010 International Conference on Computer Application and System Modeling (ICCASM 2010)*, Vol. 11. V11–161–V11–165. https://doi.org/10.1109/ICCASM.2010.5623237

[12] Timothy M. Chan. 2007. More Algorithms for All-Pairs Shortest Paths in Weighted Graphs. In *Proceedings of the Thirty-Ninth Annual ACM Symposium on Theory of Computing* (San Diego, California, USA) *(STOC '07)*. Association for Computing Machinery, New York, NY, USA, 590–598. https://doi.org/10.1145/1250790.1250877

[13] Stephen Chou, Fredrik Kjolstad, and Saman Amarasinghe. 2018. Format Abstraction for Sparse Tensor Algebra Compilers. *Proc. ACM Program. Lang.* 2, OOPSLA, Article 123 (oct 2018), 30 pages. https://doi.org/10.1145/3276493

[14] Stijn Dongen. 2000. Graph Clustering by Flow Simulation. *PhD thesis, Center for Math and Computer Science (CWI)* (05 2000).

[15] Iain S. Duff, Michael A. Heroux, and Roldan Pozo. 2002. An Overview of the Sparse Basic Linear Algebra Subprograms: The New Standard from the BLAS Technical Forum. *ACM Trans. Math. Softw.* 28, 2 (jun 2002), 239–267. https://doi.org/10.1145/567806.567810

[16] D. K. Faddeev and V. N. Faddeeva. 1981. Computational methods of linear algebra. *Journal of Soviet Mathematics* 15, 5 (1981), 531–650. https://doi.org/10.1007/BF01086544

[17] Mirko Farina, Usman Ahmad, Ahmad Taha, Hussein Younes, Yusuf Mesbah, Xiao Yu, and Witold Pedrycz. 2024. Sparsity in transformers: A systematic literature review. *Neurocomputing* 582 (2024), 127468. https://doi.org/10.1016/j.neucom.2024.127468

[18] Trevor Gale, Matei Zaharia, Cliff Young, and Erich Elsen. 2020. Sparse GPU Kernels for Deep Learning. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis* (Atlanta, Georgia) *(SC '20)*. IEEE Press, Article 17, 14 pages. https://dl.acm.org/doi/10.5555/3433701.3433723

[19] John R. Gilbert, Steve Reinhardt, and Viral B. Shah. 2006. High-Performance Graph Algorithms from Parallel Sparse Matrices. In *Proceedings of the 8th International Conference on Applied Parallel Computing: State of the Art in Scientific Computing* (Umeå, Sweden) *(PARA'06)*. Springer-Verlag, Berlin, Heidelberg, 260–269. https://link.springer.com/chapter/10.1007/978-3-540-75755-9_32

[20] John R. Gilbert, Steve Reinhardt, and Viral B. Shah. 2008. A Unified Framework for Numerical and Combinatorial Computing. *Computing in Science & Engineering* 10, 2 (March 2008), 20–25. https://doi.org/10.1109/MCSE.2008.45

[21] Branko Grünbaum and Geoffrey C. Shephard. 1987. *Tilings and Patterns*. W. H. Freeman & Co., New York.

[22] Fred G. Gustavson. 1978. Two Fast Algorithms for Sparse Matrices: Multiplication and Permuted Transposition. *ACM Trans. Math. Softw.* 4, 3 (sep 1978), 250–269. https://doi.org/10.1145/355791.355796

[23] Václav Hapla, David Horák, and Michal Merta. 2012. Use of Direct Solvers in TFETI Massively Parallel Implementation. In *Proceedings of the 11th International Conference on Applied Parallel and Scientific Computing* (Helsinki, Finland) *(PARA'12)*. Springer-Verlag, Berlin, Heidelberg, 192–205. https://doi.org/10.1007/978-3-642-36803-5_14

[24] Kartik Hegde, Hadi Asghari-Moghaddam, Michael Pellauer, Neal Crago, Aamer Jaleel, Edgar Solomonik, Joel Emer, and Christopher W. Fletcher. 2019. ExTensor: An Accelerator for Sparse Tensor Algebra. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture* (Columbus, OH, USA) *(MICRO '52)*. Association for Computing Machinery, New York, NY, USA, 319–333. https://doi.org/10.1145/3352460.3358275

[25] Eun-Jin Im, Katherine Yelick, and Richard Vuduc. 2004. Sparsity: Optimization Framework for Sparse Matrix Kernels. *The International Journal of High Performance Computing Applications* 18, 1 (2004), 135–158. https://doi.org/10.1177/1094342004041296 arXiv:https://doi.org/10.1177/1094342004041296

[26] Valentin Isaac--Chassande, Adrian Evans, Yves Durand, and Frédéric Rousseau. 2024. Dedicated Hardware Accelerators for Processing of Sparse Matrices and Vectors: A Survey. *ACM Trans. Archit. Code Optim.* 21, 2, Article 27 (Feb. 2024), 26 pages. https://doi.org/10.1145/3640542

[27] Valentin Isaac--Chassande, Adrian Evans, Yves Durand, and Frédéric Rousseau. 2024. SpDCache: Region-Based Reduction Cache for Outer-Product Sparse Matrix Kernels. In *2024 IEEE 35th International Conference on Application-specific Systems, Architectures and Processors (ASAP)*. IEEE Computer Society, Los Alamitos, CA, USA, 3–7. https://doi.org/10.1109/ASAP61560.2024.00012

[28] Satoshi Itoh, Pablo Ordejón, and Richard M. Martin. 1995. Order-N tight-binding molecular dynamics on parallel computers. *Computer Physics Communications* 88, 2 (1995), 173–185. https://doi.org/10.1016/0010-4655(95)00031-A

[29] Jeremy Kepner, David Bader, Aydın Buluç, John Gilbert, Timothy Mattson, and Henning Meyerhenke. 2015. Graphs, Matrices, and the GraphBLAS: Seven Good Reasons. *Procedia Computer Science* 51, International Conference On Computational Science, ICCS 2015 (2015), 2453–2462. https://doi.org/10.1016/j.procs.2015.05.353

[30] David R. Kincaid, Thomas C. Oppe, and David M. Young. 1989. *ITPACKV 2D user's guide*. Technical Report. United States. https://doi.org/10.2172/7093021

[31] Vladimir Kiriansky, Yunming Zhang, and Saman Amarasinghe. 2016. Optimizing Indirect Memory References with Milk. In *Proceedings of the 2016 International Conference on Parallel Architectures and Compilation* (Haifa, Israel) *(PACT '16)*. Association for Computing Machinery, New York, NY, USA, 299–312. https://doi.org/10.1145/2967938.2967948

[32] Fredrik Kjolstad, Shoaib Kamil, Stephen Chou, David Lugato, and Saman Amarasinghe. 2017. The Tensor Algebra Compiler. *Proc. ACM Program. Lang.* 1, OOPSLA, Article 77 (oct 2017), 29 pages. https://doi.org/10.1145/3133901

[33] Scott P. Kolodziej, Mohsen Aznaveh, Matthew Bullock, Jarrett David, Timothy A. Davis, Matthew Henderson, Yifan Hu, and Read Sandstrom. 2019. The SuiteSparse Matrix Collection Website Interface. *Journal of Open Source Software* 4, 35 (2019), 1244. https://doi.org/10.21105/joss.01244

[34] Monica D. Lam, Edward E. Rothberg, and Michael E. Wolf. 1991. The Cache Performance and Optimizations of Blocked Algorithms. *SIGPLAN Not.* 26, 4 (apr 1991), 63–74. https://doi.org/10.1145/106973.106981

[35] Ruipeng Li, Björn Sjögreen, and Ulrike Meier Yang. 2021. A New Class of AMG Interpolation Methods Based on Matrix-Matrix Multiplications. *SIAM Journal on Scientific Computing* 43, 5 (2021), S540–S564. https://doi.org/10.1137/20M134931X

[36] J.-C. Luo. 1992. Algorithms for Reducing the Bandwidth and Profile of a Sparse Matrix. *Computers & Structures* 44, 3 (1992), 535–548. https://doi.org/10.1016/0045-7949(92)90386-E

[37] Liviu Mafteiu-Scai. 2014. The Bandwidths of a Matrix. A Survey of Algorithms. *West University of Timisoara Annals, Timisoara, Romania* LII (12 2014), 183– 223. https://doi.org/10.2478/awutm-2014-0019

[38] Kiran Matam, Siva Rama Krishna Bharadwaj Indarapu, and Kishore Kothapalli. 2012. Sparse matrix-matrix multiplication on modern architectures. In *2012 19th International Conference on High Performance Computing*. 1–10. https://doi.org/10.1109/HiPC.2012.6507483

[39] Anurag Mukkara, Nathan Beckmann, and Daniel Sanchez. 2019. PHI: Architectural Support for Synchronization- and Bandwidth-Efficient Commutative Scatter Updates. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture* (Columbus, OH, USA) *(MICRO '52)*. Association for Computing Machinery, New York, NY, USA, 1009–1022. https://doi.org/10.1145/3352460.3358254

[40] Subhankar Pal, Jonathan Beaumont, Dong-Hyeon Park, Aporva Amarnath, Siying Feng, Chaitali Chakrabarti, Hun-Seok Kim, David Blaauw, Trevor Mudge, and Ronald Dreslinski. 2018. OuterSPACE: An Outer Product Based Sparse Matrix Multiplication Accelerator. In *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. Institute of Electrical and Electronics Engineers, New York, NY, USA, 724–736. https://doi.org/10.1109/HPCA.2018.00067

[41] Gerald Penn. 2006. Efficient transitive closure of sparse matrices over closed semirings. *Theoretical Computer Science* 354, 1 (2006), 72–81. https://doi.org/10.1016/j.tcs.2005.11.008 Algebraic Methods in Language Processing.

[42] Michael O Rabin and Vijay V Vazirani. 1989. Maximum matchings in general graphs through randomization. *Journal of Algorithms* 10, 4 (1989), 557–567. https://doi.org/10.1016/0196-6774(89)90005-9

[43] Oleg I. Ryabkov. 2022. Implementation of the Algebraic Multigrid Solver Designed for Graphics Processing Units Based on the AMGCL Framework. In *Parallel Computational Technologies*, Leonid Sokolinsky and Mikhail Zymbler (Eds.), Vol. 1618. Springer International Publishing, Cham, 131–142. https://doi.org/10.1007/978-3-031-11623-0_10

[44] Yousef Saad. 2003. *Iterative Methods for Sparse Linear Systems* (second ed.). Society for Industrial and Applied Mathematics. https://doi.org/10.1137/1.9780898718003

[45] Sriseshan Srikanth, Anirudh Jain, Thomas M. Conte, Erik P. Debenedictis, and Jeanine Cook. 2021. SortCache: Intelligent Cache Management for Accelerating Sparse Data Workloads. *ACM Trans. Archit. Code Optim.* 18, 4, Article 56 (sep 2021), 24 pages. https://doi.org/10.1145/3473332

[46] Nitish Srivastava, Hanchen Jin, Jie Liu, David Albonesi, and Zhiru Zhang. 2020. MatRaptor: A Sparse-Sparse Matrix Multiplication Accelerator Based on Row-Wise Product. In *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. Institute of Electrical and Electronics Engineers, New York, NY, USA, 766–780. https://doi.org/10.1109/MICRO50266.2020.00068

[47] Vivienne Sze, Yu-Hsin Chen, Tien-Ju Yang, and Joel Emer. 2020. *Efficient Processing of Deep Neural Networks* (1 ed.). Springer Cham. https://doi.org/10.1007/978-3-031-01766-7

[48] Reginald P. Tewarson. 1973. *Sparse matrices*. Vol. 69. Academic press New York, State University of New York, Stony Brook, NY.

[49] Richard Vuduc, James W Demmel, and Katherine A Yelick. 2005. OSKI: A library of automatically tuned sparse matrix kernels. *Journal of Physics: Conference Series* 16, 1 (jan 2005), 521. https://doi.org/10.1088/1742-6596/16/1/071

[50] F. Vázquez, E M Garzon, Jose Martinez, and J Fernández. 2009. The sparse matrix vector product on GPUs. *Proceedings of the 2009 International Conference on Computational and Mathematical Methods in Science and Engineering* 2 (01 2009).

[51] Chen Wang, Chuan Xu, and Abdel Lisser. 2014. Bandwidth Minimization Problem. In *10th International Conference on MOdeling, Optimization and SIMulation - MOSIM14*. Nancy, France. https://hal.science/hal-01166658

[52] Lucas Wilkinson, Kazem Cheshmi, and Maryam Mehri Dehnavi. 2023. Register Tiling for Unstructured Sparsity in Neural Network Inference. *Proc. ACM Program. Lang.* 7, PLDI, Article 188 (June 2023), 26 pages. https://doi.org/10.1145/3591302

[53] Yannan Nellie Wu, Po-An Tsai, Angshuman Parashar, Vivienne Sze, and Joel S. Emer. 2022. Sparseloop: An Analytical Approach To Sparse Tensor Accelerator Modeling. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. Institute of Electrical and Electronics Engineers and Association for Computing Machinery, New York, NY, USA, 1377–1395. https://doi.org/10.1109/MICRO56248.2022.00096

[54] Wm. A. Wulf and Sally A. McKee. 1995. Hitting the Memory Wall: Implications of the Obvious. *SIGARCH Comput. Archit. News* 23, 1 (mar 1995), 20–24. https://doi.org/10.1145/216585.216588

[55] Ichitaro Yamazaki and Xiaoye S. Li. 2011. On Techniques to Improve Robustness and Scalability of a Parallel Hybrid Linear Solver. In *High Performance Computing for Computational Science – VECPAR 2010*, José M. Laginha M. Palma, Michel Daydé, Osni Marques, and João Correia Lopes (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 421–434.

[56] Chao Zhang, Maximilian Bremer, Cy Chan, John Shalf, and Xiaochen Guo. 2022. ASA: Accelerating Sparse Accumulation in Column-Wise SpGEMM. *ACM Trans. Archit. Code Optim.* 19, 4, Article 49 (sep 2022), 24 pages. https://doi.org/10.1145/3543068

[57] Guowei Zhang, Nithya Attaluri, Joel S. Emer, and Daniel Sanchez. 2021. Gamma: Leveraging Gustavson's Algorithm to Accelerate Sparse Matrix Multiplication. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems* (Virtual, USA) *(ASPLOS '21)*. Association for Computing Machinery, New York, NY, USA, 687–701. https://doi.org/10.1145/3445814.3446702

Contents

Plan (Brouillon)

- Introduction **(~3 p.)**
  - "light" > give the entry point on sparse matrices and irregular accesses **(1-2 p.)**
  - Mains contributions **(1 p.)**
  - introduce the structure of the manuscript
- Background **(~5 p.)**
  - irregular accesses -> def, issues (in memory but not only?), domains/applications (including sparse matrix processing) **(1 p.)**
  - Focus on sparse matrices, context: it is widely used, in many domains, sparse, large, structures
  - Sparse matrices in a kernel: show where the accesses are regular vs irregular (infos on basic sparse formats (other sparse formats in appendix x), tiling, basic kernels, basic algos...) **(2-3 p.)**
  - Hardware challenges clearly defined (gather, scatter...) **(1 p.)**
- SoA **(~11 p.)**
  - Present solutions in SW for CPU/GPU, such as special sparse formats, possibility to play on algo (gust analysis), tiling methods adapted for sparse data (specific sparse format + specific algos) (expliquer clairement que ces solutions sont interressantes et pourquoi on se concentre sur le HW) **(1-2 p.)**
  - Present solutions in HW (dedicated accelerators) -> comparison table along common characteristics **(1-2 p.)**
  - Presentation of the major accelerators (selection of 3 important: processor assist PHI, standalone GAMMA, multi algo SPADA, ACES) (others in appendix xi) **(3-4 p.)**
  - Problem of comparison / comparison of standalone / other analysis (issue of matrix structure) -> Objectives: convince reader that SoA was done in depth, and acknoledge the matrice structure issue **(1 p.)**
  - Takeaways -> paths for designing an accelerator (end od SoA paper) **(1-2 p.)**
  - We choose a path -> show which one (coarse grain, at the level of the SoA, comparable to SortCache, PHI...)
- High level architectural presentation of SpDCache **(~10 p.)**
  - Reminder of the path we choose with more details: cache / reductions / OP SpMV / ... **(1 p.)**
  - Emphasis the comparison between concerned SoA accelerators -> show what is already done and useful, show what is missing
  - Present the case of banded matrices as an example of structure to leverage **(1-2 p.)**
  - SpDCache, architecture presentation with two regions, details on REDs transactions, IBRC vs OBRT **(7 p.)**
  - Initial results from Python model (ASAP) and note that a probabilistic model and RTL model will follow
- Mathematical analysis of the cache memory accesses for processing sparse matrices **(~23 p.)**
  - Explain the objectives of this chapter (~formal justification of the concept + cache size/region dimensioning) **(1 p.)**
  - Describe the design space of the path we choose (with reminders if necessary) -> introduce base parameters and/or variables, give the exact algorithm... **(1 p.)**
  - Show the interest of separate cache regions for A/b and c (formal) -> show that a GC is obviously not efficient when random accesses / confirm why SortCache, PHI, ... exist **(2 p.)**

    - Show that A and b are not a limiting factor with a formal reasoning -> give the needed space for GRC when prefetcher (clear statement that gather is not the aim of the thesis) **(2-3 p.)**
    - We use a GRC (no modification) for A and b -> get nb loads
    - Show the interest of a reduction cache region for c -> REDs / reduce the number of mem accesses / link to SoA (see ASAP) **(2 p.)**
    - We created a probabilistic model of the reduction cache which is based on the following principles ... (presented in detail in appendix n) / avantage of this model / we implemented this model in C / validation of the model with state of the art article on generic cache formalisation / introduce the graphs for the next sections **(1-2 p.)**
    - Show that reduction cache does not handle randomness -> model graph **(1-2 p.)**
    - Show RedC 1reg vs RedC 2reg on banded example **(2-3 p.)**
    - Show best cache dimensions for the dense part (band) **(1 p.)**
    - Show that sparse region (out of band) could use an other region for fine grain density (case of sub bands) -> dimensions **(1-2 p.)**
    - Show that sparse region could use adapted HW because of the isolated elements -> dimensions **(1-2 p.)**
    - Show how it would work with a uniform matrix -> how does it affects the dimensions **(1 p.)**
    - Conclude on our solution and dimensioning **(1 p.)**
- Micro architecture of SpDCache **(~10 p.)**
    - Present our hardware solution, based on the concept
    - Start with basic info on HPDcache (only the one that are useful for SpDCache)
    - Describe modif in pipeline / hazard management / throughput
    - Modif in RAM accesses ...
    - Stats: git / bugs / lines of code / synthesis
- RTL simulation results **(~10 p.)**
    - Experimentations: RTL simulation results -> matrices chosen, generated / metrics / ...
    - Results / comparison with theory / analysis / conclusion
- Future enhancement **(~8 p.)**
    - Architecture presentation with three regions (add OBRC, CB...) **(2 p.)**
    - Architecture of eviction anticipation **(2 p.)**
    - Present multicore version -> scale **(2 p.)**
    - Present additional optimisation (region config, learn from first iteration) **(1-2 p.)**
- Conclusions **(~2 p.)**
    - ..
- Appendices
    - All sparse formats **(~5 p.)**
    - Other accelerators of SoA **(~10 p.)**
    - Probabilistic model of the reduction cache **(~10 p.)**
    - Python model **(~1 p.)**